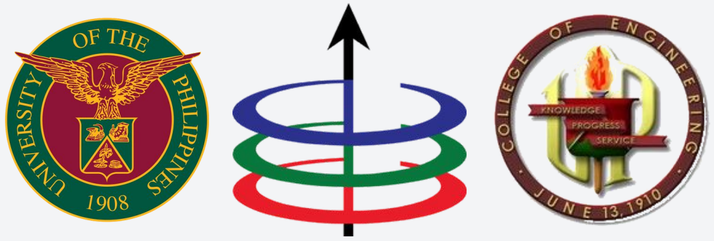


Lecture 3
Analyzing Recursive Functions

EEE 121 2s2526

Steven S. Sison

- Recursion Overview
- Divide-and-Conquer Algorithms
- Recursive Backtracking

- **Recursion Overview**
- Divide-and-Conquer Algorithms
- Recursive Backtracking

Iterative algorithms are slow af

On our previous discussion of sorting algorithms, we saw that an iterative approach to sorting is very inefficient having a **worst time complexity of $O(n^2)$!**

Recursion, recursion, recursion, base case :D

To make faster algorithms, we often employ **recursion**.

- Recursion is a **repeated application** of a procedure.
- A recursive function is a function defined in terms of itself.

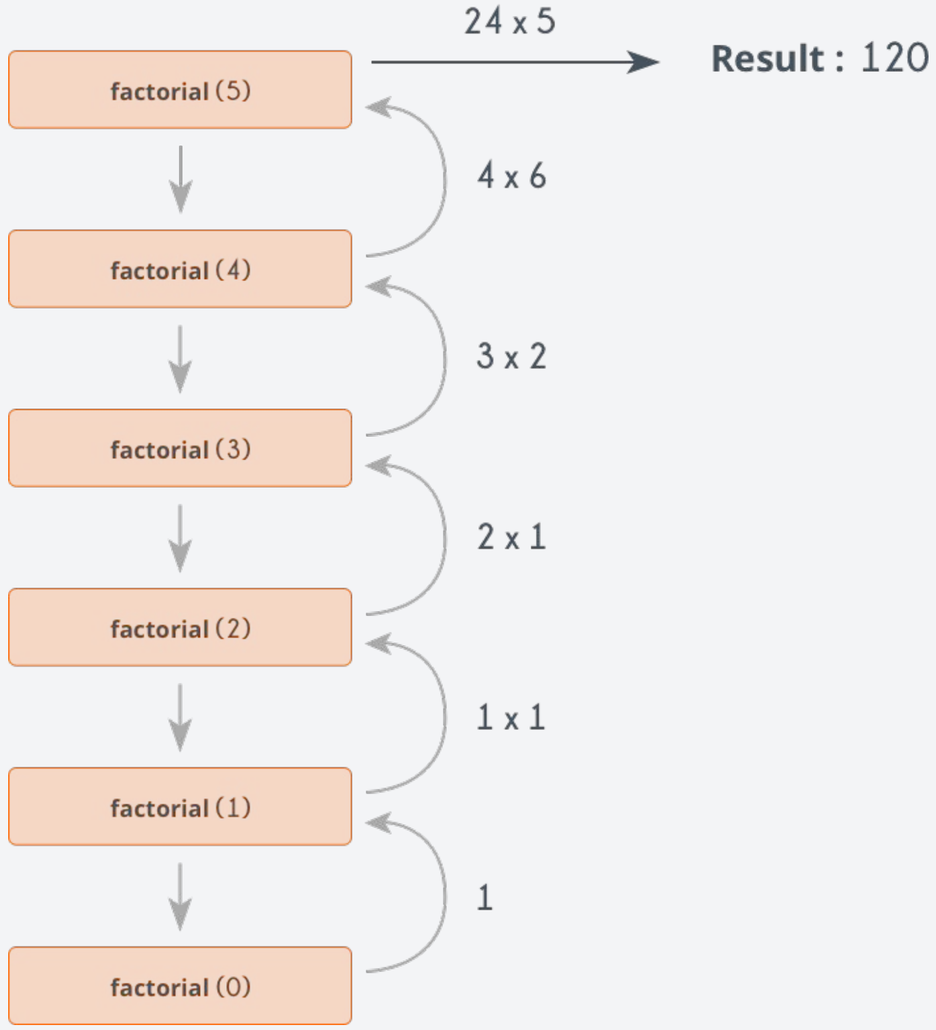
Recursion, recursion, recursion, base case :D

A recursive function has two components:

- **Base Case**: a trivial case that can be solved without recursion
- **Recursive Call**: a progression towards the base case

Example: factorial

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n - 1 );
   }
}
```



Recursion, recursion, recursion, base case :D

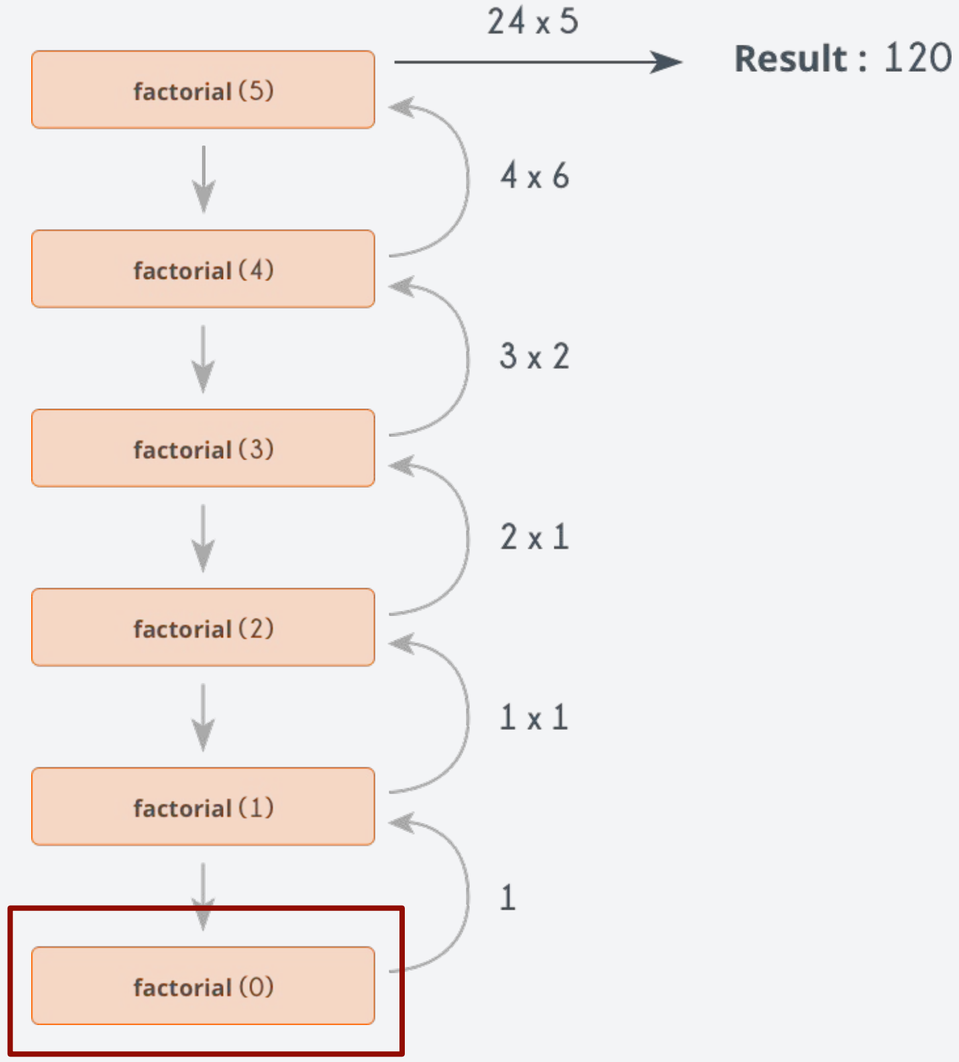
A recursive function has two components:

- **Base Case**: a trivial case that can be solved without recursion
- **Recursive Call**: a progression towards the base case

Example: factorial

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n - 1 );
   }
}
```

This is the base case



Recursion, recursion, recursion, base case :D

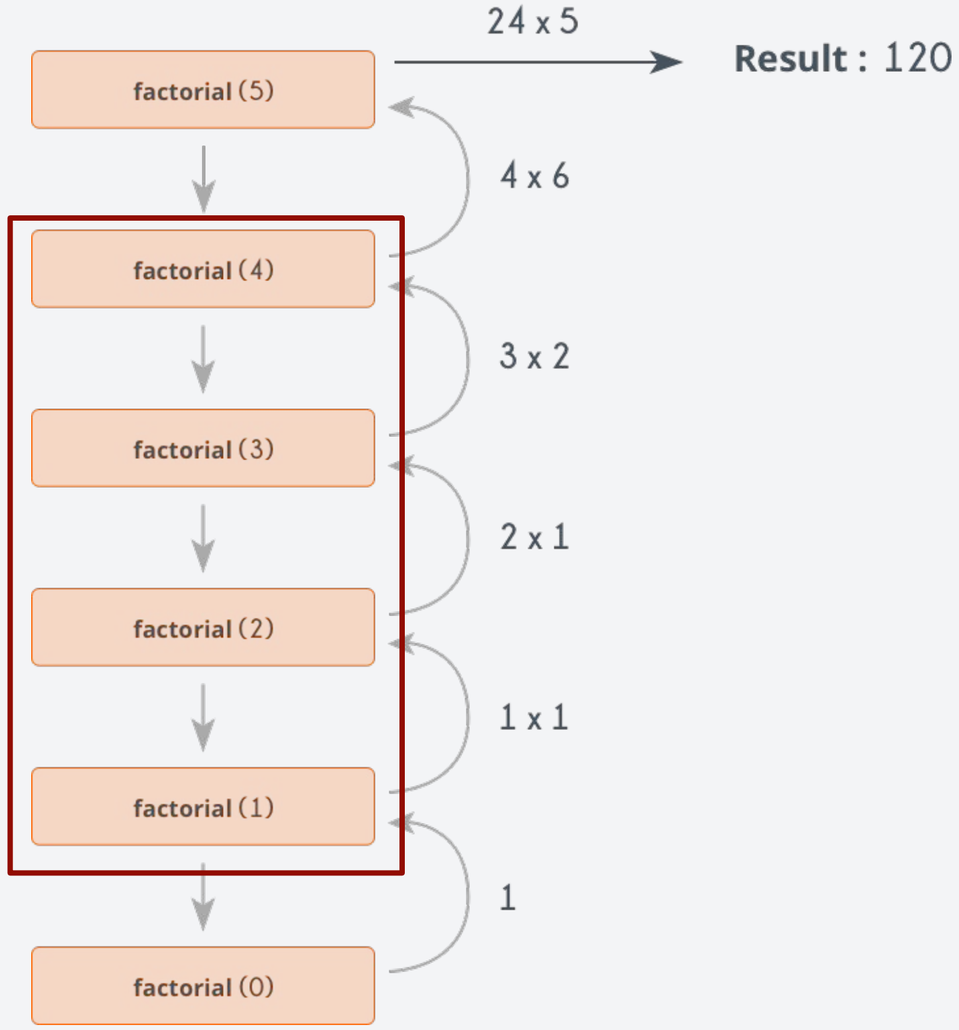
A recursive function has two components:

- **Base Case**: a trivial case that can be solved without recursion
- **Recursive Call**: a progression towards the base case

Example: factorial

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n - 1 );
   }
}
```

These are the recursive calls



Recursion, recursion, recursion, base case :D

A recursive function has two components:

- **Base Case**: a trivial case that can be solved without recursion
- **Recursive Call**: a progression towards the base case

Recursion is cool, but you have to mind the base case. Otherwise, your code will not end!

But when is recursion applicable to use?

Why did we use recursion in solving for the factorial of a number? Why not just do it iteratively?

Recursive

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n – 1 );
   }
}
```

Iterative

```
int factorial( int n ) {
   int result = 1;
   for (int i = 2; i <= n; i++) {
      result *= i;
   }
   return result;
}
```

But when is recursion applicable to use?

Why did we use recursion in solving for the factorial of a number? Why not just do it iteratively?

Recursive

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n – 1 );
   }
}
```

Iterative

```
int factorial( int n ) {
   int result = 1;
   for (int i = 2; i <= n; i++) {
      result *= i;
   }
   return result;
}
```

- Recursive algorithms often provide a more cleaner and intuitive code. Less loop and state management.

But when is recursion applicable to use?

Why did we use recursion in solving for the factorial of a number? Why not just do it iteratively?

Recursive

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n – 1 );
   }
}
```

Iterative

```
int factorial( int n ) {
   int result = 1;
   for (int i = 2; i <= n; i++) {
      result *= i;
   }
   return result;
}
```

- Recursive algorithms are often ideal when there are a lot of repeating terms that call itself. For example, $fact(n) = n(fact(n-1)) = n(n-1)(fact(n-2)) = ...$

But when is recursion applicable to use?

Why did we use recursion in solving for the factorial of a number? Why not just do it iteratively?

Recursive

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n – 1 );
   }
}
```

Iterative

```
int factorial( int n ) {
   int result = 1;
   for (int i = 2; i <= n; i++) {
      result *= i;
   }
   return result;
}
```

- Some algorithms such as divide-and-conquer and data structures such as trees naturally fit a recursive structure.

But when is recursion applicable to use?

Why did we use recursion in solving for the factorial of a number? Why not just do it iteratively?

Recursive

```
int factorial( int n ) {
   if ( n <= 1 ) {
      return 1;
   } else {
      return n * factorial( n – 1 );
   }
}
```

Iterative

```
int factorial( int n ) {
   int result = 1;
   for (int i = 2; i <= n; i++) {
      result *= i;
   }
   return result;
}
```

- Recursion can sometimes provide a better worst time complexity than iterative algorithms, but sometimes at the cause of more memory overhead (trade-offs).

How do we analyze recursive algorithms?

To analyze the runtime of a recursive algorithm, we often use **recurrence relations**.

- It is an equation that is defined by itself, similar to what a recursion is.

How do we analyze recursive algorithms?

To analyze the runtime of a recursive algorithm, we often use **recurrence relations**.

- It is an equation that is defined by itself, similar to what a recursion is.

Types of recurrence relations:

1. **Decrease-by-one recurrence** - $T(n) = T(n-1) + f(n)$
2. **Decrease-by-constant-factor recurrence** - $T(n) = aT(n/b) + f(n)$

How do we analyze recursive algorithms?

To analyze the runtime of a recursive algorithm, we often use **recurrence relations**.

- It is an equation that is defined by itself, similar to what a recursion is.

Types of recurrence relations:

1. **Decrease-by-one recurrence** - $T(n) = T(n-1) + f(n)$
2. **Decrease-by-constant-factor recurrence** - $T(n) = aT(n/b) + f(n)$

Solving the recurrence relation can be done by using either of these methods:

1. Substitution (or unrolling) - expands the recurrence relation step-by-step
2. Recursion Trees - visualizes the recursive call
3. Master's Theorem - used for divide-and-conquer algorithms

Example 1: `factorial(n)`

Going back to our example, we can see that the factorial function that we created are defined as:

$$n! = fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = \ldots$$

Example 1: `factorial(n)`

Going back to our example, we can see that the factorial function that we created are defined as:

$$n! = fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = \ldots$$

If we let T(n) to be the runtime of our factorial function, we can see that it can be represented as follows.

$$T(n) = \begin{cases} c_0 & , \quad n \leq 1 \\ T(n-1) + c_1 & , \ otherwise \end{cases}$$

Example 1: `factorial(n)`

Going back to our example, we can see that the factorial function that we created are defined as:

$$n! = fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = \ldots$$

If we let T(n) to be the runtime of our factorial function, we can see that it can be represented as follows.

This is the base case that returns 1 in O(1) time.

$$T(n) = \begin{cases} c_0 & , \quad n \leq 1 \\ T(n-1) + c_1 & , \ otherwise \end{cases}$$

Example 1: `factorial(n)`

Going back to our example, we can see that the factorial function that we created are defined as:

$$n! = fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = \ldots$$

If we let T(n) to be the runtime of our factorial function, we can see that it can be represented as follows.

This is the base case that returns 1 in O(1) time.

$$T(n) = \begin{cases} c_0 & , \quad n \leq 1 \\ T(n-1) + c_1 & , \ otherwise \end{cases}$$

This is the time complexity of the recursive call

Example 1: `factorial(n)`

Going back to our example, we can see that the factorial function that we created are defined as:

$$n! = fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = \ldots$$

If we let T(n) to be the runtime of our factorial function, we can see that it can be represented as follows.

This is the base case that returns 1 in O(1) time.

$$T(n) = \begin{cases} c_0 & , \quad n \leq 1 \\ T(n-1) + c_1 & , \ otherwise \end{cases}$$

This is the time complexity of the multiplication in O(1) time

This is the time complexity of the recursive call

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \qquad \forall n > 1$$

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

$$T(n) = T(n-k) + \sum_{i=1}^{k} c_1 \quad \forall n > 1$$

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

$$T(n) = T(n-k) + \sum_{i=1}^{k} c_1 \quad \forall n > 1$$

For k = n-1,

$$T(n) = T(1) + (n-1)c_1 = O(n)$$

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

$$T(n) = T(n-k) + \sum_{i=1}^{k} c_1 \quad \forall n > 1$$

For k = n-1,

$$T(n) = T(1) + (n-1)c_1 = O(n)$$

Why n-1? We want to reach the base case at n-k = 1

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

$$T(n) = T(n-k) + \sum_{i=1}^{k} c_1 \quad \forall n > 1$$

For k = n-1,

$$T(n) = \boxed{T(1)} + (n-1)c_1 = O(n)$$

This is the base case

Example 1: `factorial(n)`

To solve for T(n), we are going to use the **substitution method**. If we expand T(n), we have:

$$T(n) = T(n-1) + c_1 \quad \forall n > 1$$

$$T(n) = T(n-2) + c_1 + c_1 \quad \forall n > 1$$

$$T(n) = T(n-3) + c_1 + c_1 + c_1 \quad \forall n > 1$$

There is a pattern!

$$T(n) = T(n-k) + \sum_{i=1}^{k} c_1 \quad \forall n > 1$$

For k = n-1,

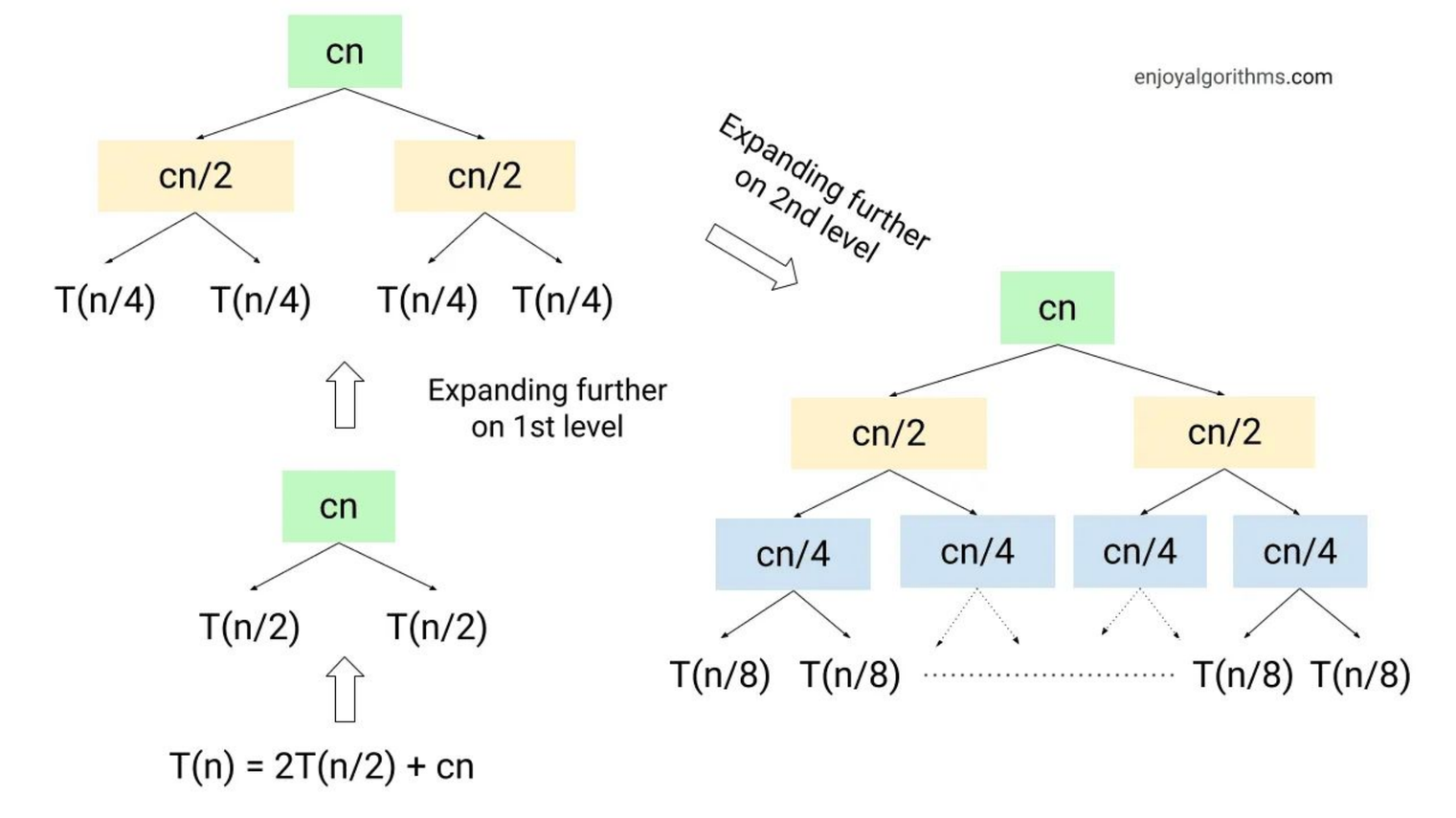
$$T(n) = \boxed{T(1)} + \boxed{(n-1)c_1} = O(n)$$

This is the base case

The function is recursively called n-1 times.

Recursion Trees (Not required, but good to know!)

Recursion trees are useful for visualizing the recursive call of our function. It is also an intuitive way to determine time complexity.

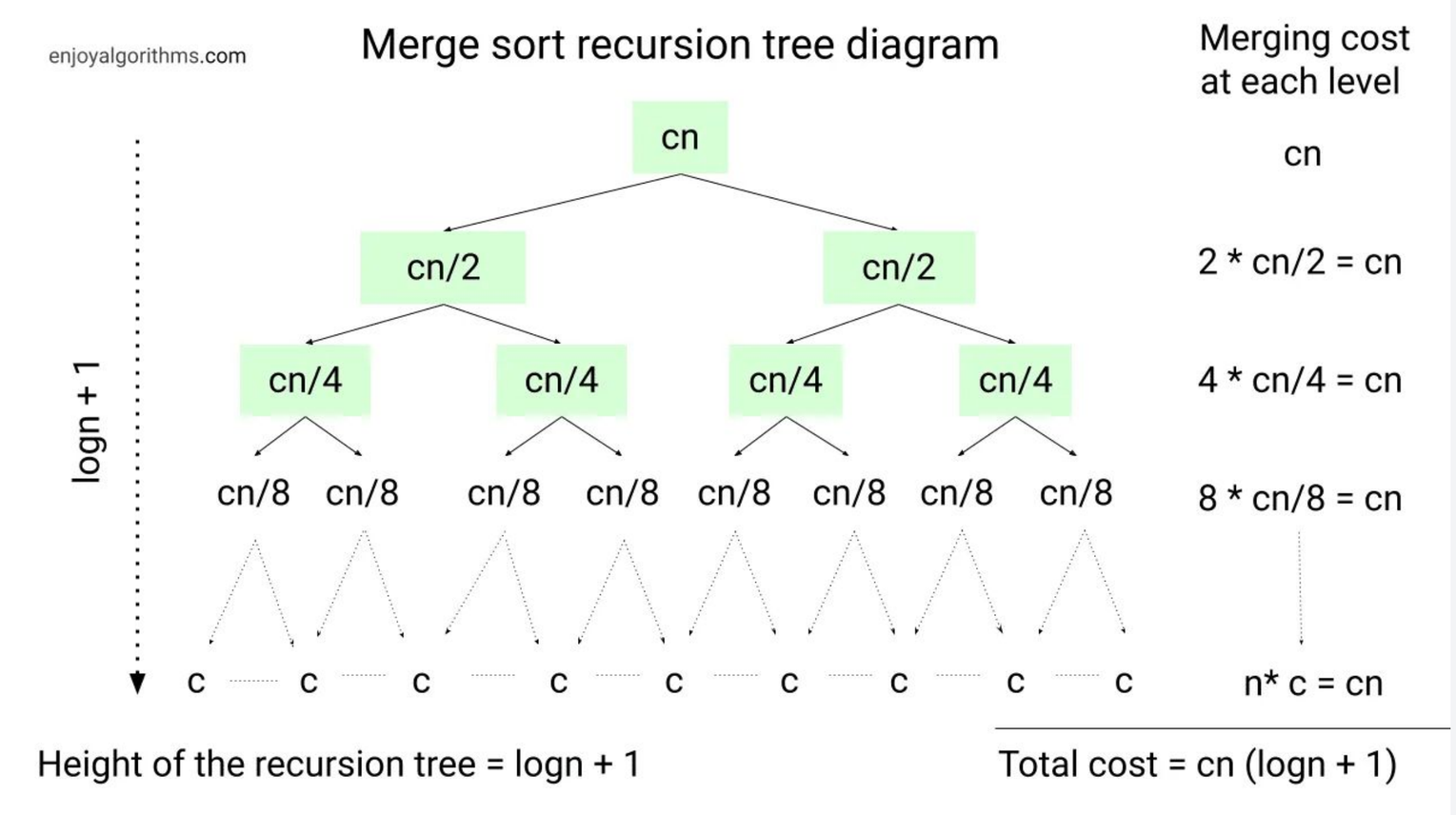


The time complexity is solved by getting the sum of the work done per level.

- In this example, the cost per level is cn.

Recursion Trees (Not required, but good to know!)

Recursion trees are useful for visualizing the recursive call of our function. It is also an intuitive way to determine time complexity.



The time complexity is solved by getting the sum of the work done per level.

- In this example, the cost per level is cn.
- The total time complexity is the product of the cost per level and number of levels.

Master's Theorem (Not required, but good to know!)

Another way of solving a recurrence relation is through the Master's theorem. It is commonly used for divide-and-conquer algorithms.

Combining the three cases above gives us the following "master theorem".

Theorem 1 *The recurrence*

$$\begin{aligned} T(n) &= aT(n/b) + cn^k \\ T(1) &= c, \end{aligned}$$

where a, b, c, and k are all constants, solves to:

$$\begin{aligned} T(n) &\in \Theta(n^k) \text{ if } a < b^k \\ T(n) &\in \Theta(n^k \log n) \text{ if } a = b^k \\ T(n) &\in \Theta(n^{\log_b a}) \text{ if } a > b^k \end{aligned}$$

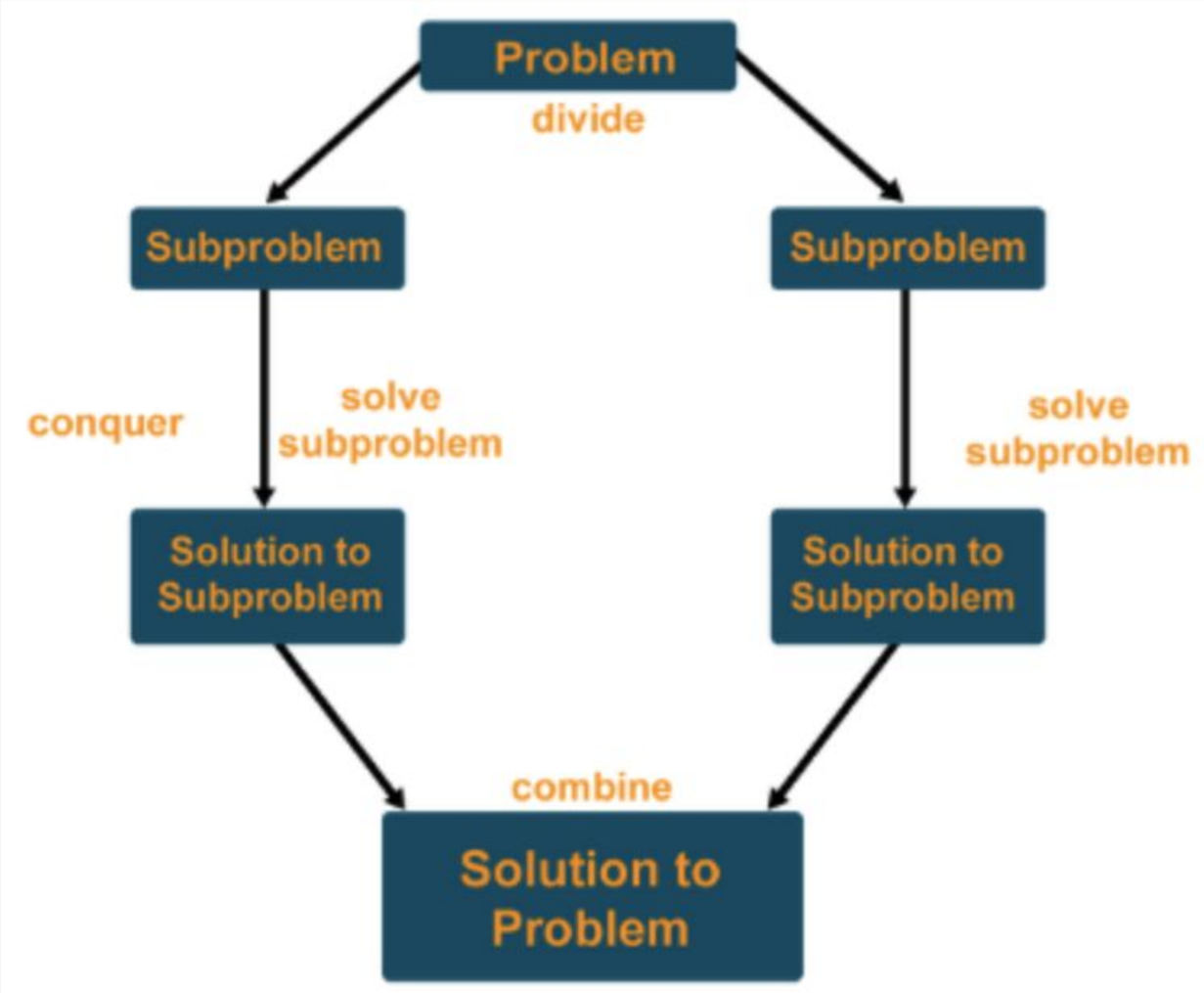
Later, we will be proving sorting algorithms with the substitution method. But you can also try it with the Master's Theorem and you'll have the same answer!

- Recursion Overview
- **Divide-and-Conquer Algorithms**
- Recursive Backtracking

Maghimay, ..., at ipasa mo

A common trend in recursive algorithms is to use it in divide-and-conquer algorithms. It works by:

- **Dividing**: Break the problem into smaller sub-problems
- **Conquer**: Solve the smaller problems and collect the results to solve the current problem

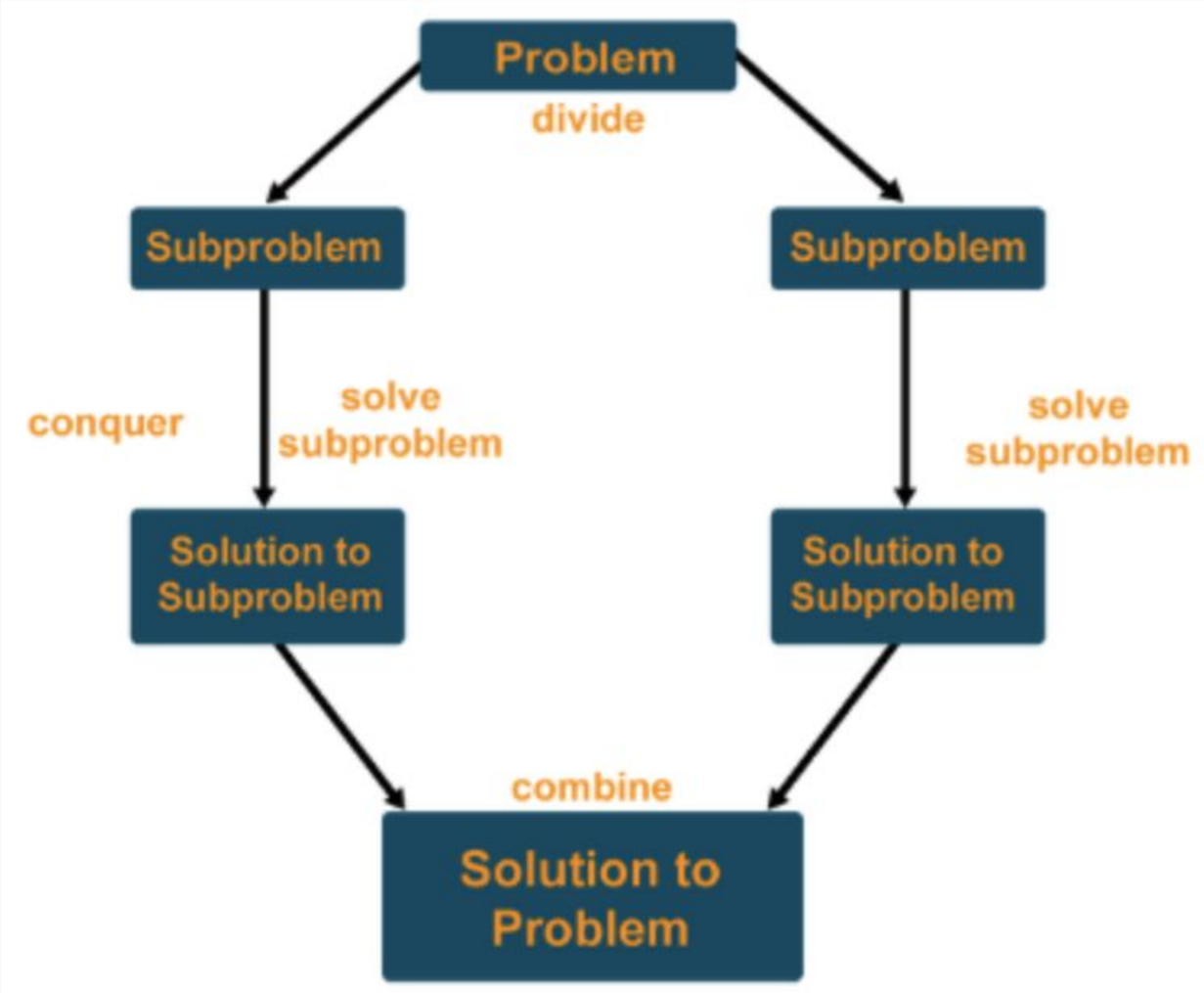


Maghimay, ..., at ipasa mo

A common trend in recursive algorithms is to use it in divide-and-conquer algorithms. It works by:

- **Dividing**: Break the problem into smaller sub-problems
- **Conquer**: Solve the smaller problems and collect the results to solve the current problem

How do you think is recursion included in the mix?

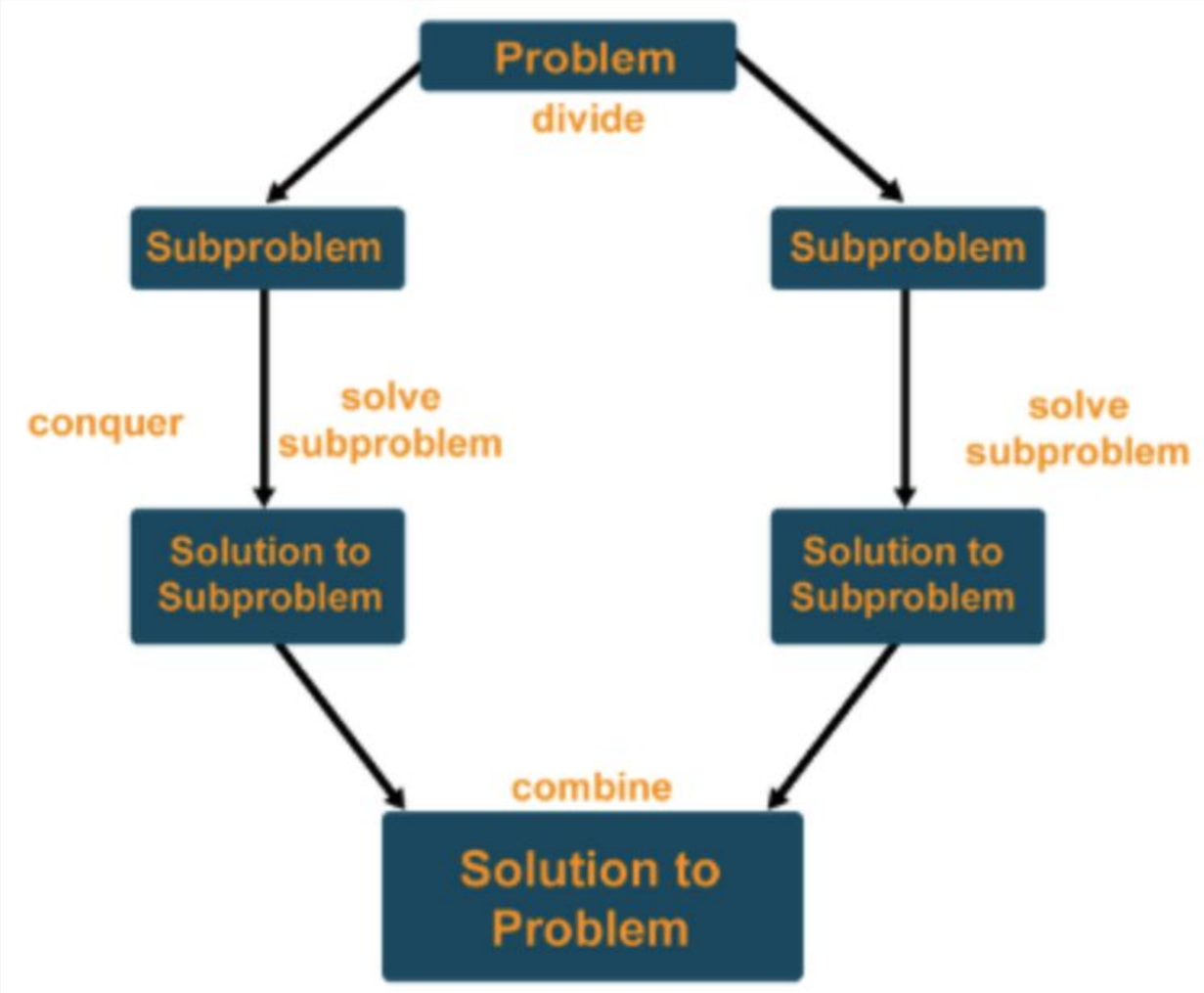


Maghimay, ..., at ipasa mo

A common trend in recursive algorithms is to use it in divide-and-conquer algorithms. It works by:

- **Dividing**: Break the problem into smaller sub-problems
- **Conquer**: Solve the smaller problems and collect the results to solve the current problem

We use the recursive call in the “conquer” part :D



Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

What can we do?

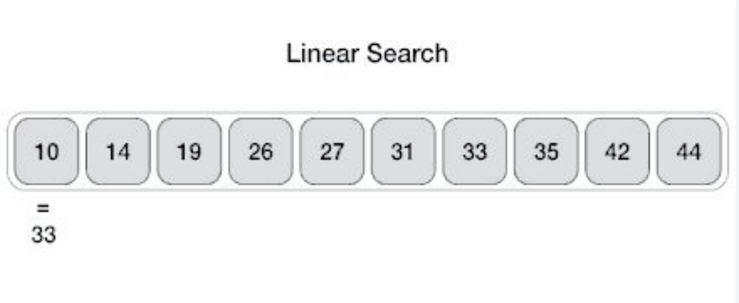
Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 1 Linear/Sequential Search

1. Start at one end of the array.
2. Look at each array element until the element is found
3. If you reach the last element and the desired element is not found, output -1.



Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 1 Linear/Sequential Search

```
int linear_search(const int *arr, int val) {
  int i = 0;
  int n = sizeof(arr)/sizeof(arr[0]);
  while(i < n && arr[i] != val) {
    i++;
  }
  if(i == n) {
    return -1;
  } else {
    return i;
  }
}
```

Time Complexity?

Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 1 Linear/Sequential Search

```
int linear_search(const int *arr, int val) {
  int i = 0;
  int n = sizeof(arr)/sizeof(arr[0]);
  while(i < n && arr[i] != val) {
    i++;
  }
  if(i == n) {
    return -1;
  } else {
    return i;
  }
}
```

Time Complexity?

O(n)

Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 1 Linear/Sequential Search

```
int linear_search(const int *arr, int val) {
  int i = 0;
  int n = sizeof(arr)/sizeof(arr[0]);
  while(i < n && arr[i] != val) {
    i++;
  }
  if(i == n) {
    return -1;
  } else {
    return i;
  }
}
```

Time Complexity?

O(n)

Let’s try to do better :D

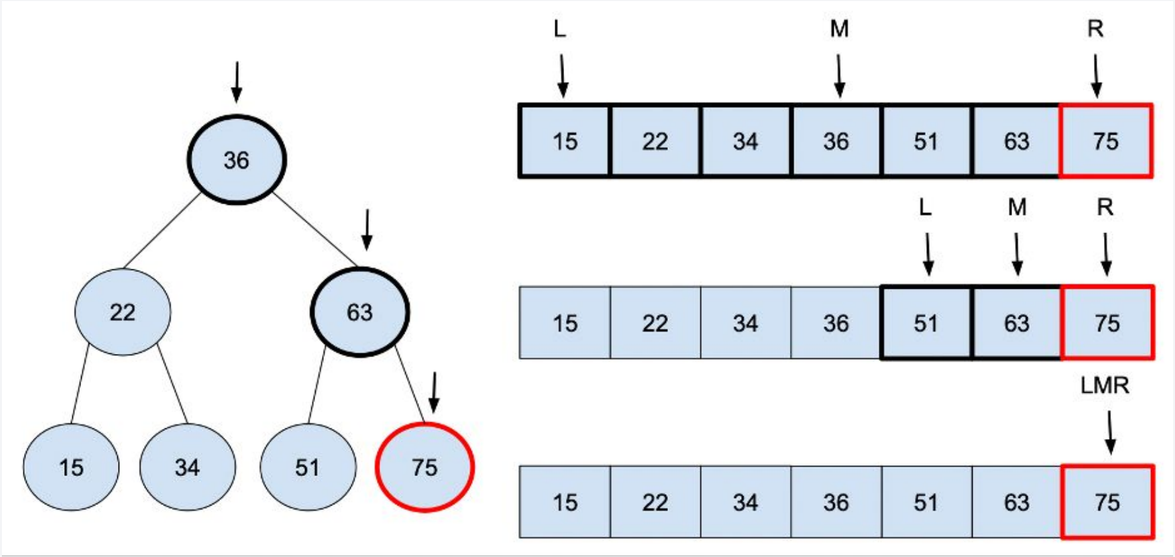
Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 2 Binary Search

1. Divide the set of items into two parts.
2. Determine which of the two parts contain the desired element, and then perform search on that part.
3. Subdivide and repeat until the element was found.



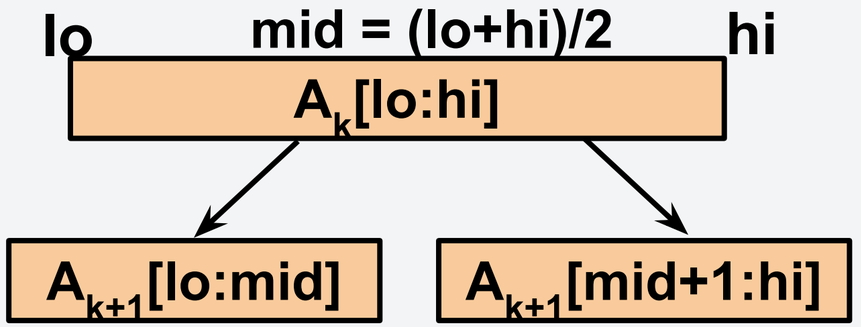
Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 2 Binary Search

```
int bin_search(int* arr, int lo, int hi, int val)
{
   int mid = (lo + hi)/2;
   if(lo == hi)
      return -1;  //not found
   if(arr[mid] == val)
      return mid; //found
   else if(arr[mid] < val) //search right
      return bin_search(arr,mid+1,hi,val);
   else // search left
      return bin_search(arr,lo,mid,val);
}
```



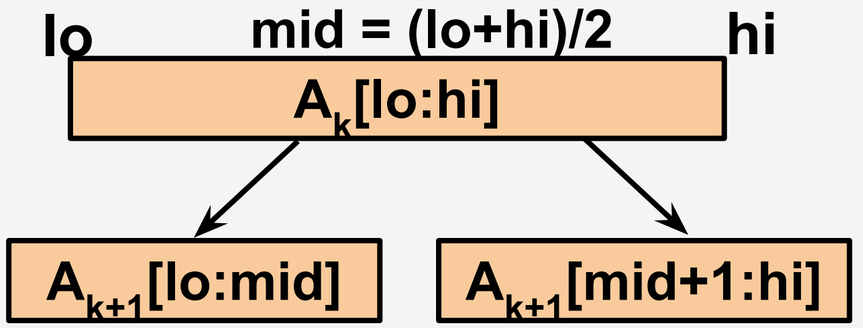
Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 2 Binary Search

```
int bin_search(int* arr, int lo, int hi, int val)
{
   int mid = (lo + hi)/2;
   if(lo == hi)
      return -1;  //not found
   if(arr[mid] == val)
      return mid; //found
   else if(arr[mid] < val) //search right
      return bin_search(arr,mid+1,hi,val);
   else // search left
      return bin_search(arr,lo,mid,val);
}
```



If the middle does not contain the desired value, we divide the problem

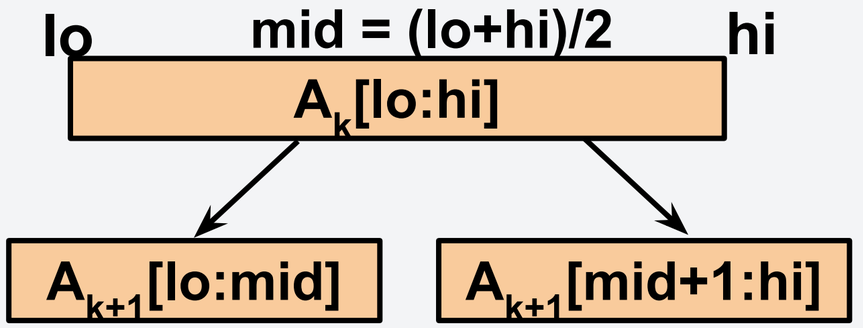
Example: Hanapin mo sa libro!

Consider a phonebook with contacts that are sorted alphabetically with n elements.

- Let A be the sorted list representing the phonebook.
- We want to find the index of some item val.

Method 2 Binary Search

```
int bin_search(int* arr, int lo, int hi, int val)
{
   int mid = (lo + hi)/2;
   if(lo == hi)
      return -1;  //not found
   if(arr[mid] == val)
      return mid; //found
   else if(arr[mid] < val) //search right
      return bin_search(arr,mid+1,hi,val);
   else // search left
      return bin_search(arr,lo,mid,val);
}
```



After dividing the array, we perform a recursive call (conquer)

Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let’s visualize it with a recursion tree!

Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let’s visualize it with a recursion tree!

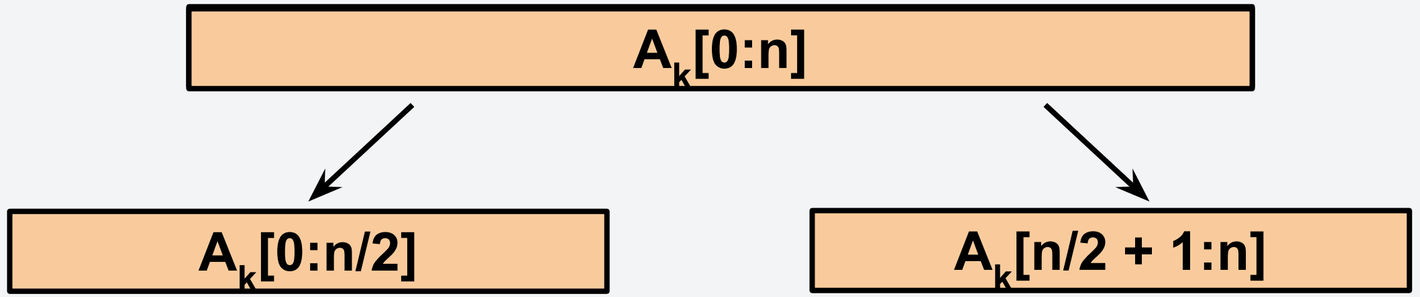
A_1: Problem size is n

Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2



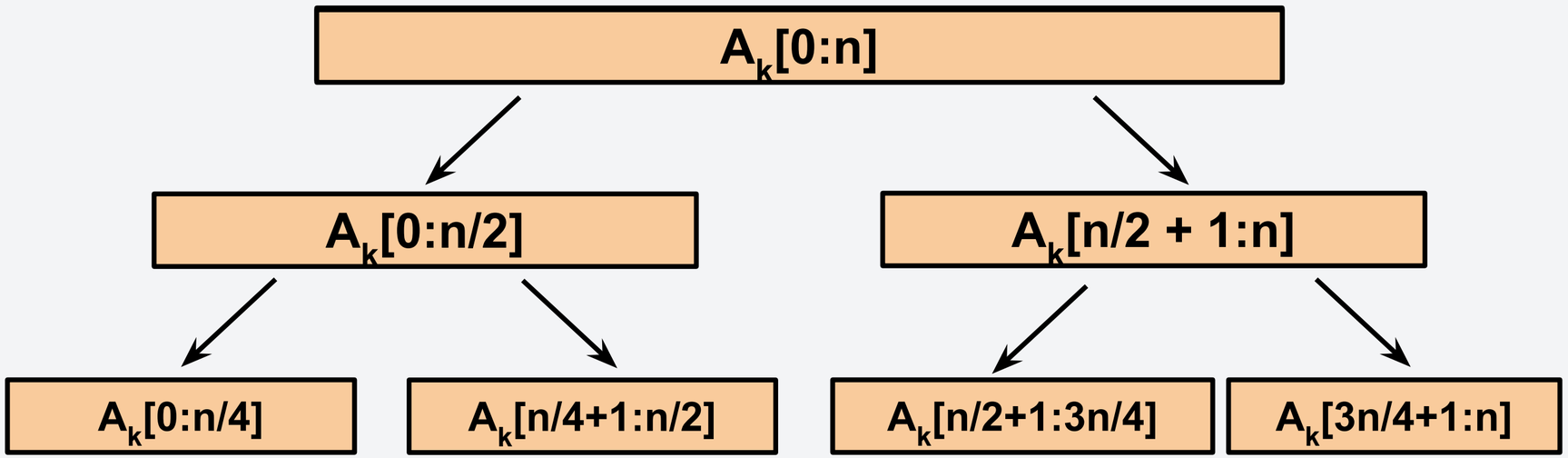
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



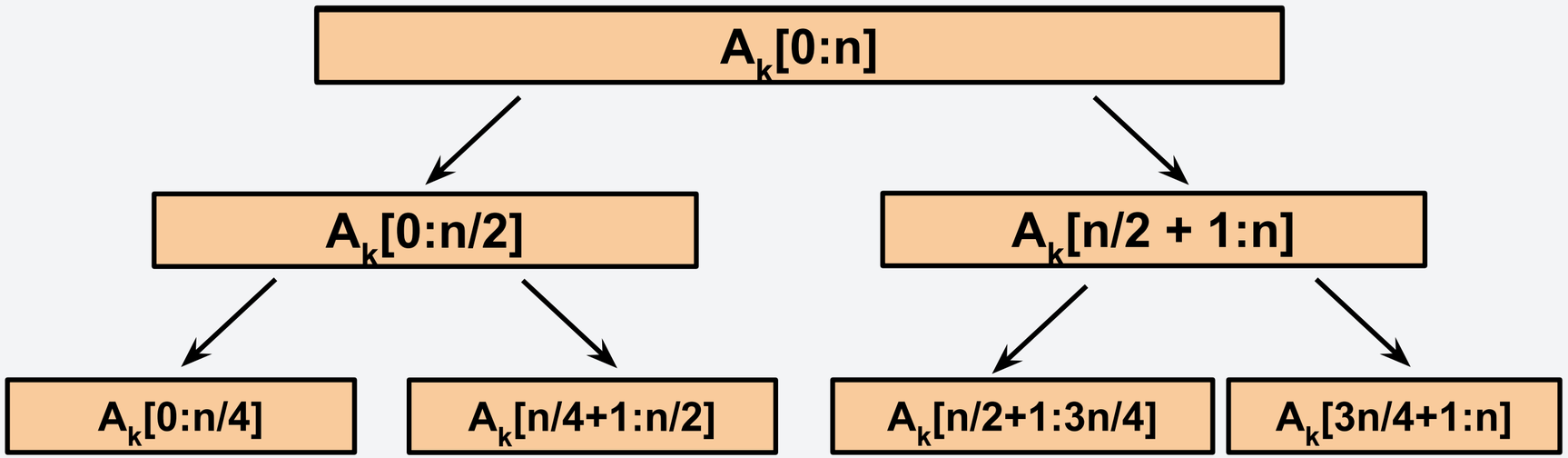
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



And so on, so forth...

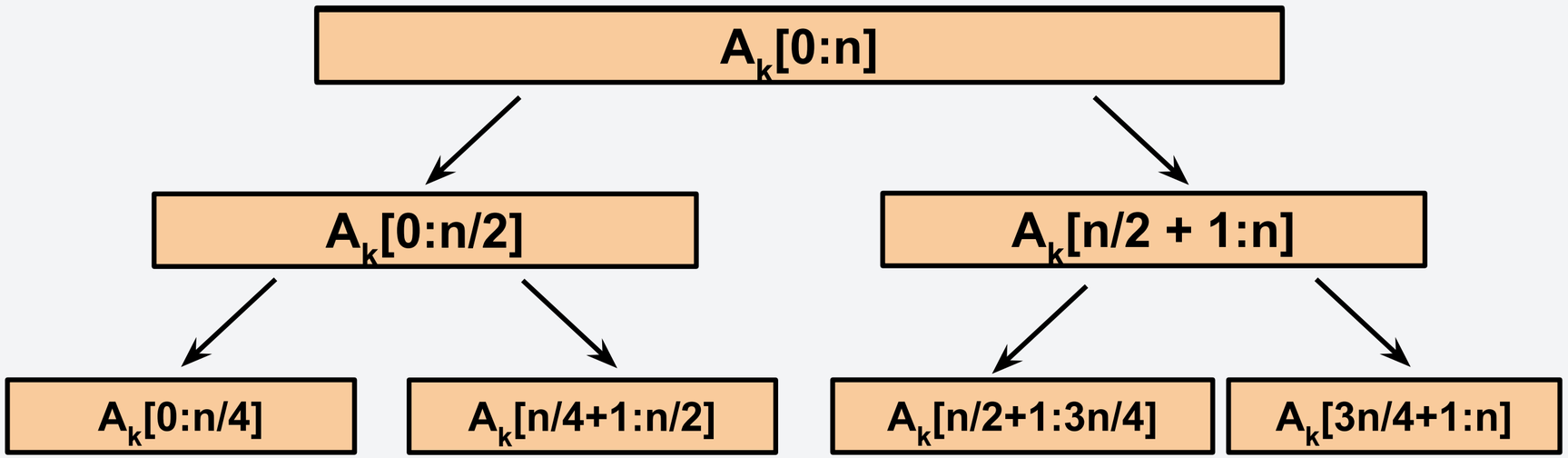
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



Using the tree, can you solve the time complexity?

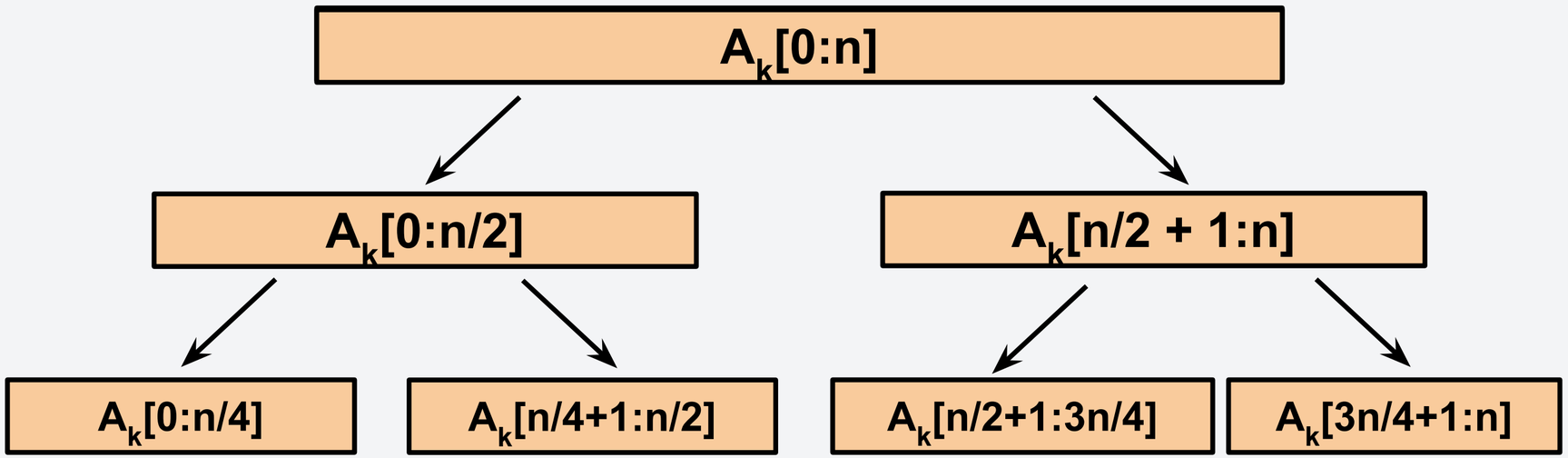
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

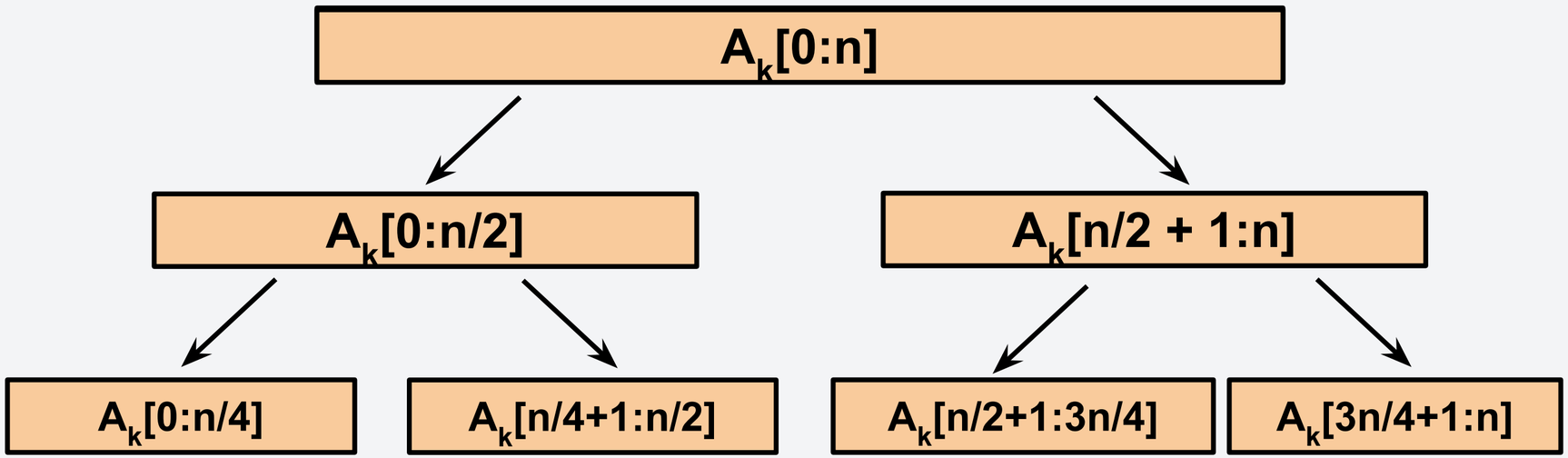
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

Why only T(n/2) not 2T(n/2)?

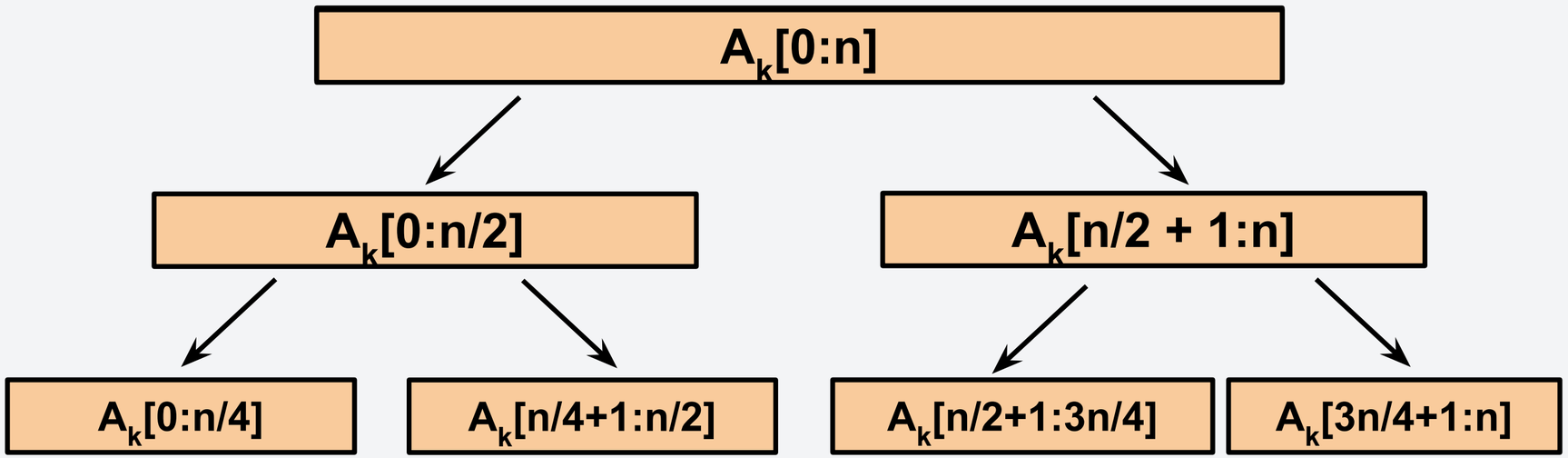
Example: Hanapin mo sa libro!

But how de we determine the time complexity? Let's visualize it with a recursion tree!

A_1: Problem size is n

A_2: Problem size is n/2

A_3: Problem size is n/4



Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

We only conquer half :)

Example: Hanapin mo sa libro!

Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

Example: Hanapin mo sa libro!

Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

Solving the first few steps,

$$\begin{aligned} T_{bin}(n) &= T_{bin}(n/2) + c_1 \\ &= T_{bin}(n/4) + c_1 + c_1 \\ &\vdots \\ &= T_{bin}\left(n/2^k\right) + c_1 k \end{aligned}$$

Example: Hanapin mo sa libro!

Recurrence Relation

$$T_{bin}(n) = \begin{cases} T_{bin}(n/2) + c_1, & n > 1 \\ c_0, & otherwise \end{cases}$$

Solving the first few steps,

$$T_{bin}(n) = T_{bin}(n/2) + c_1$$
$$= T_{bin}(n/4) + c_1 + c_1$$
$$\vdots$$
$$= T_{bin}(n/2^k) + c_1 k$$

Let n = 2^k to reach the base case,

$$T_{bin}(n) = \underbrace{T_{bin}(1)} + c_1 \cdot \log_2 n$$

This is the base case with constant time complexity

$$T_{bin}(n) = O(\log_2 n)$$

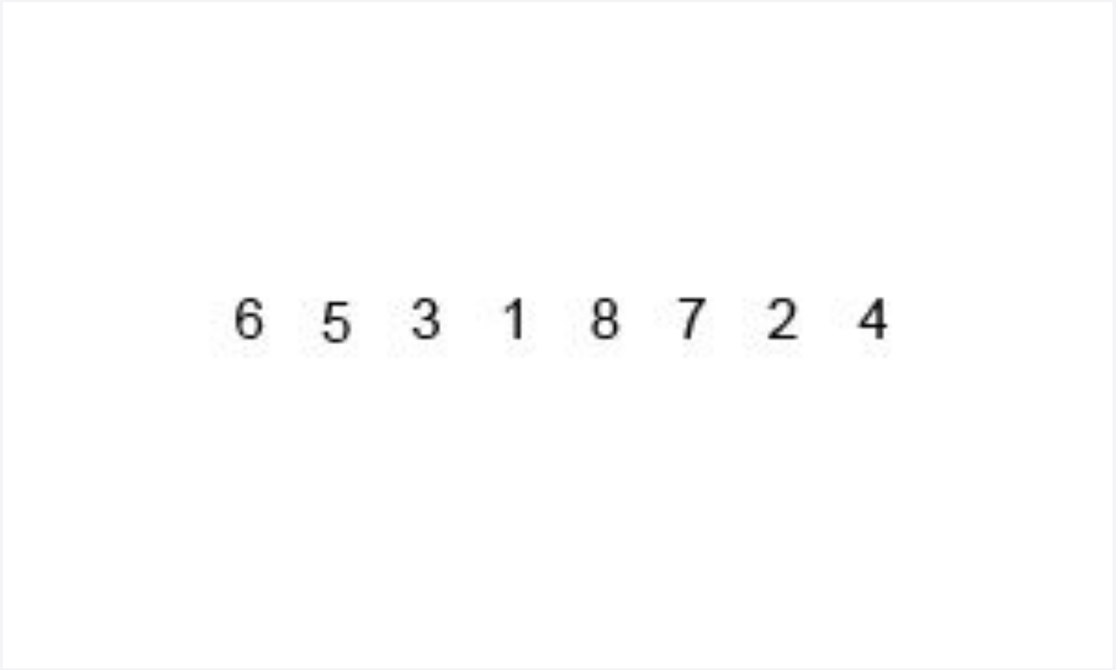
Let’s continue with the sorting algorithms

Let’s practice more divide-and-conquer algorithms by continuing with our sorting algorithms. There are two more sorting algorithms we are going to explore:

Algorithm	Proved?	Time Complexity	
		Best Case	Worst Case
Insertion Sort	Yes!	O(n)	$O(n^2)$
Bubble Sort	Yes!	O(n)	$O(n^2)$
Selection Sort	Assignment :)	$O(n^2)$	$O(n^2)$
Merge Sort	Today!		
Quick Sort			

Sorting Algorithm 4: Merge Sort

The merge sort algorithm is defined recursively. It divides the arrays smaller until there is only one element left in the sub-array and then **merges** those sub-arrays.



Pseudocode:

```
split each element into partitions of size 1
recursively merge adjacent partitions
  for i = leftPartIdx to rightPartIdx
    if leftPartHeadValue <= rightPartHeadValue
      copy leftPartHeadValue
    else: copy rightPartHeadValue; Increase InvIdx
copy elements back to original array
```

Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge_sort(A):
    if len(A) <= 1:
      return A
    L = merge_sort(A[0:n/2])
    R = merge_sort(A[n/2:n])
    return merge(L, R)
```

Continuously divide the array

until the sub-array only has one

element.

Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge_sort(A):
    if len(A) <= 1:
      return A
    L = merge_sort(A[0:n/2])
    R = merge_sort(A[n/2:n])
    return merge(L, R)
```

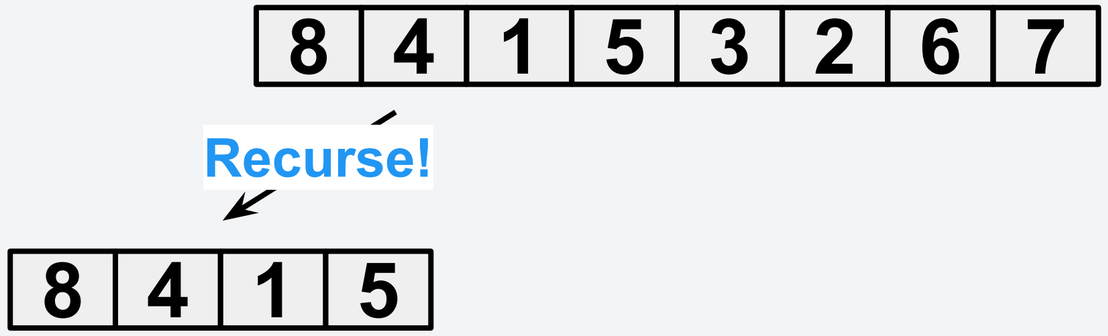
Continuously divide the array until the sub-array only has one element.

This is the **recursive call**.

Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge_sort(A):
    if len(A) <= 1:
      return A
    L = merge_sort(A[0:n/2])
    R = merge_sort(A[n/2:n])
    return merge(L, R)
```



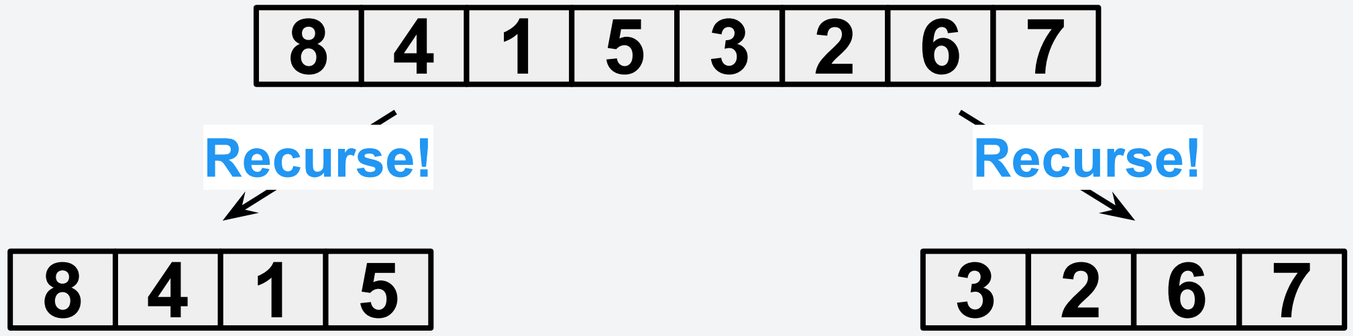
Continuously divide the array until the sub-array only has one element.

This is the **recursive call**.

Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge_sort(A):
    if len(A) <= 1:
      return A
    L = merge_sort(A[0:n/2])
    R = merge_sort(A[n/2:n])
    return merge(L, R)
```



Continuously divide the array until the sub-array only has one element.

This is the **recursive call**.

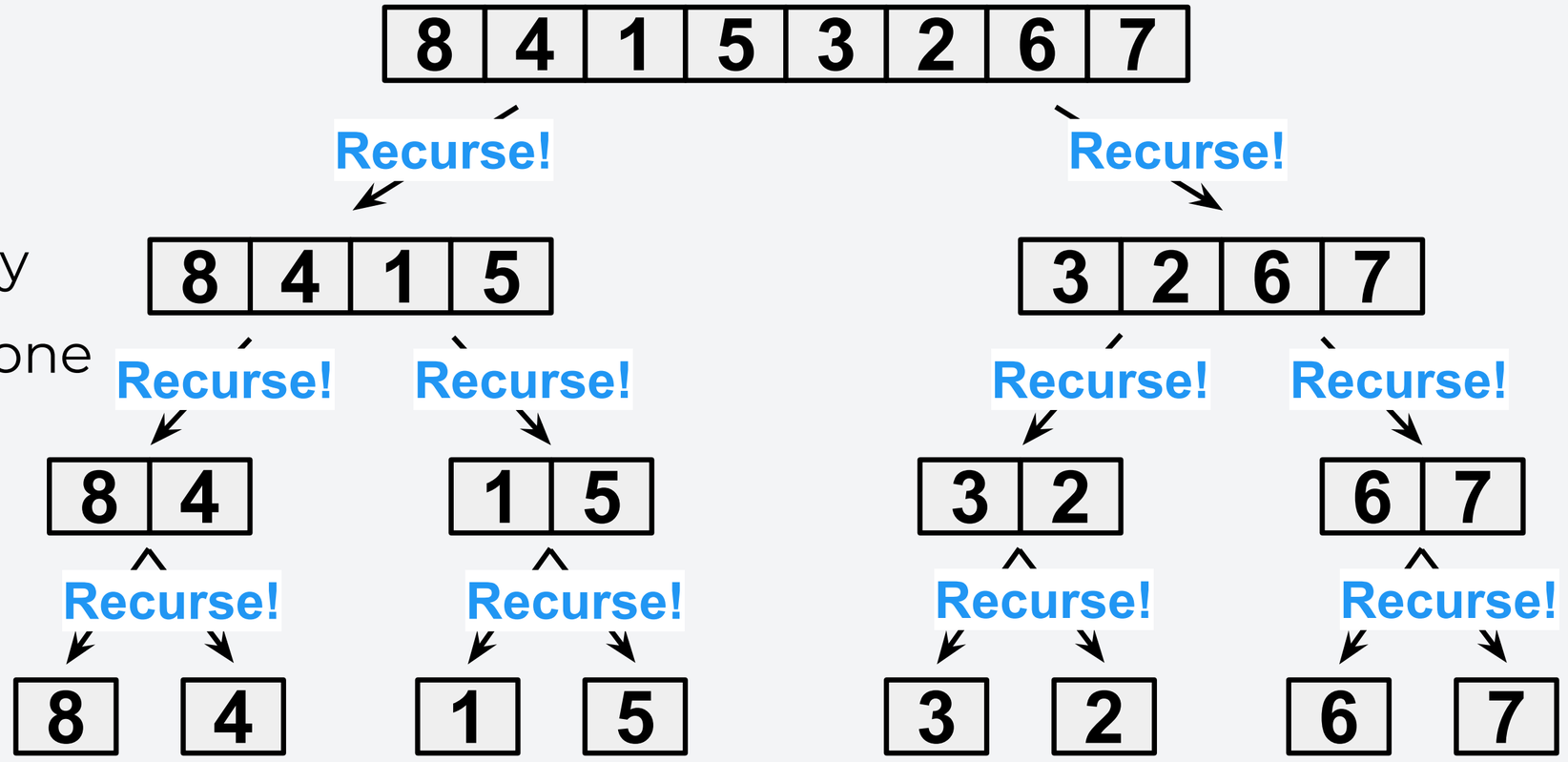
Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge_sort(A):
    if len(A) <= 1:
      return A
    L = merge_sort(A[0:n/2])
    R = merge_sort(A[n/2:n])
    return merge(L, R)
```

Continuously divide the array until the sub-array only has one element.

This is the **recursive call**.

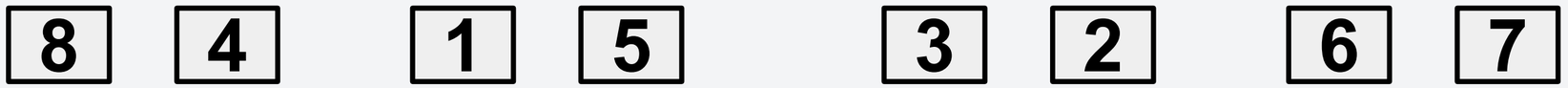


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

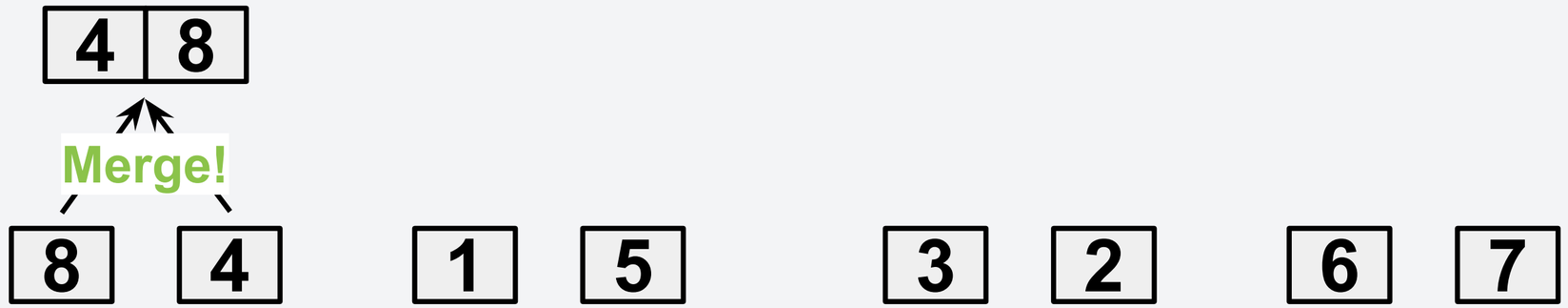


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

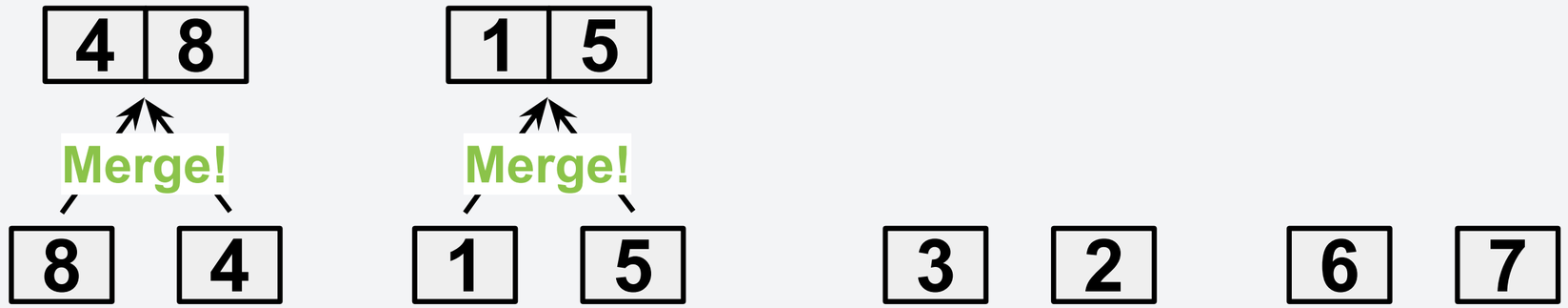


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

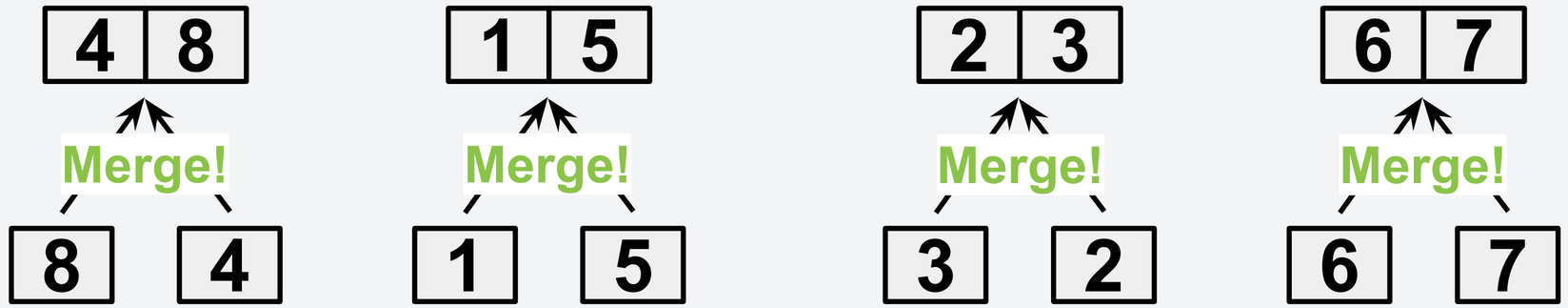


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

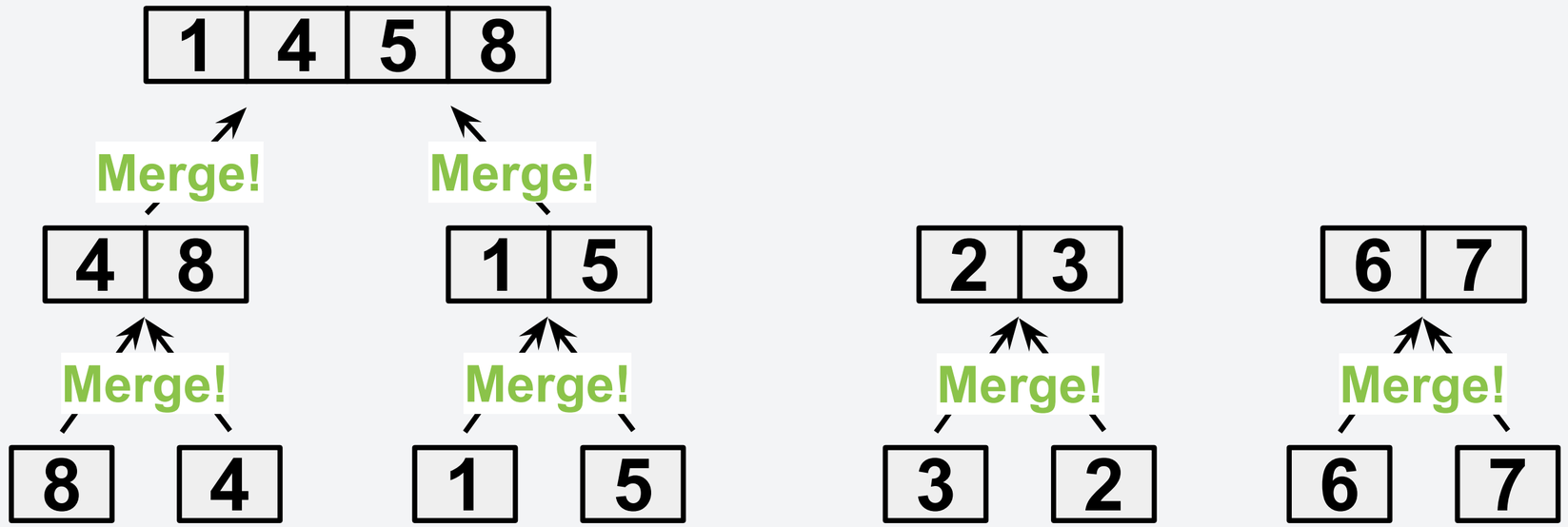


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

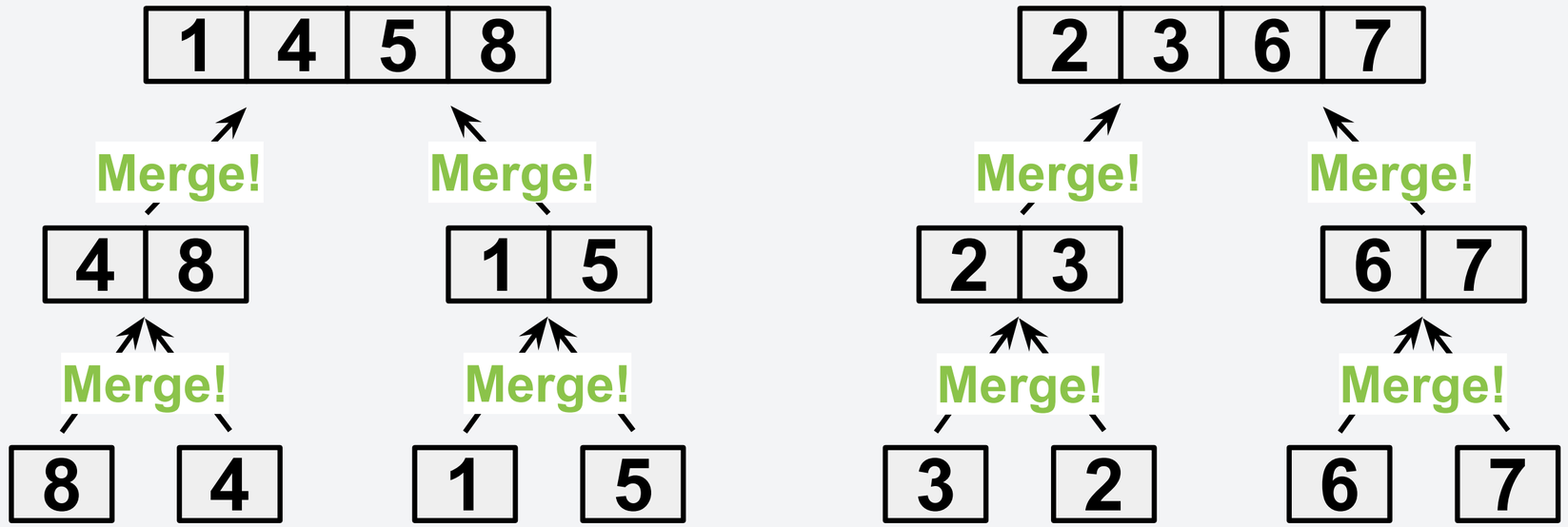


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

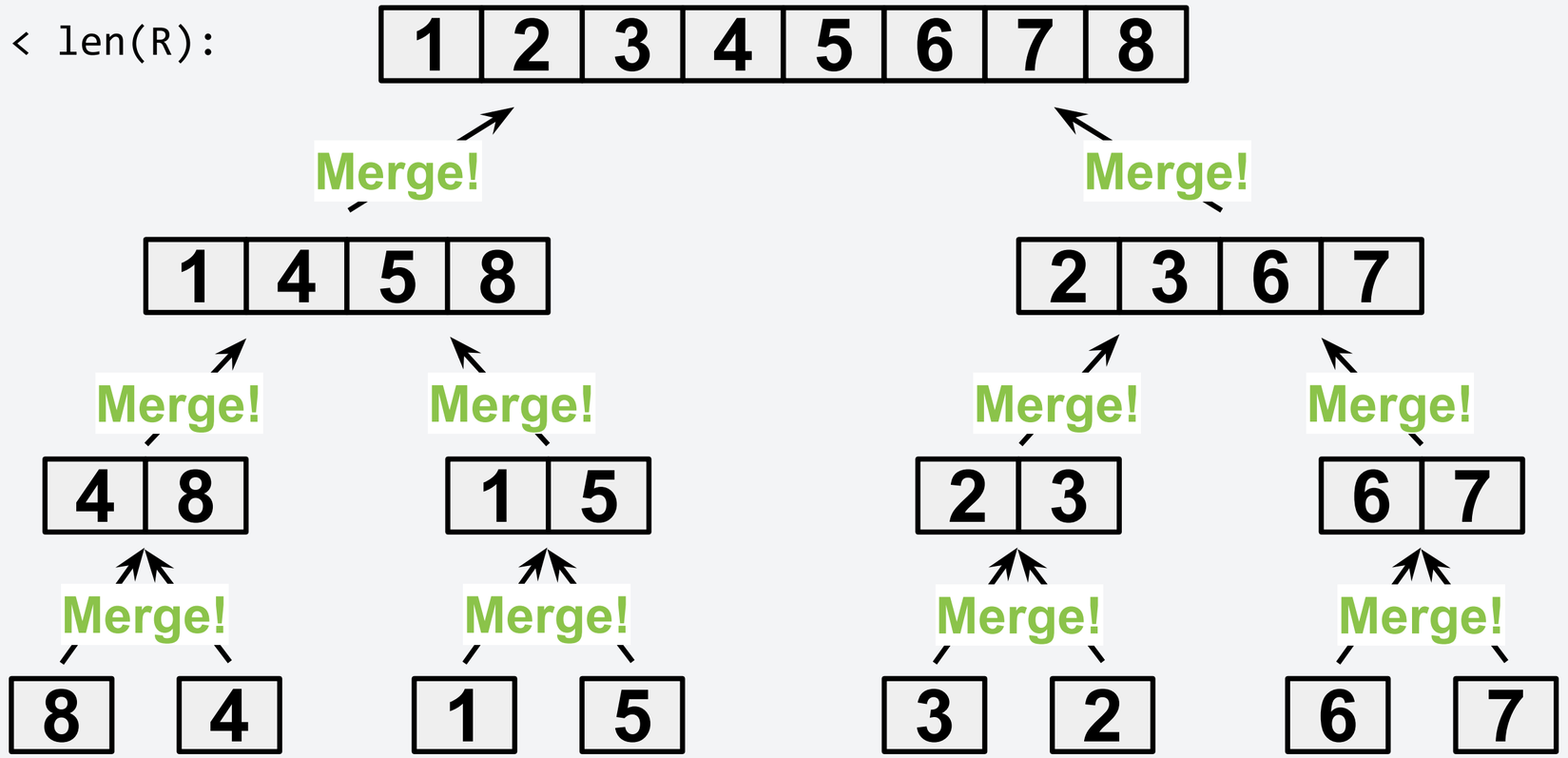


Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.



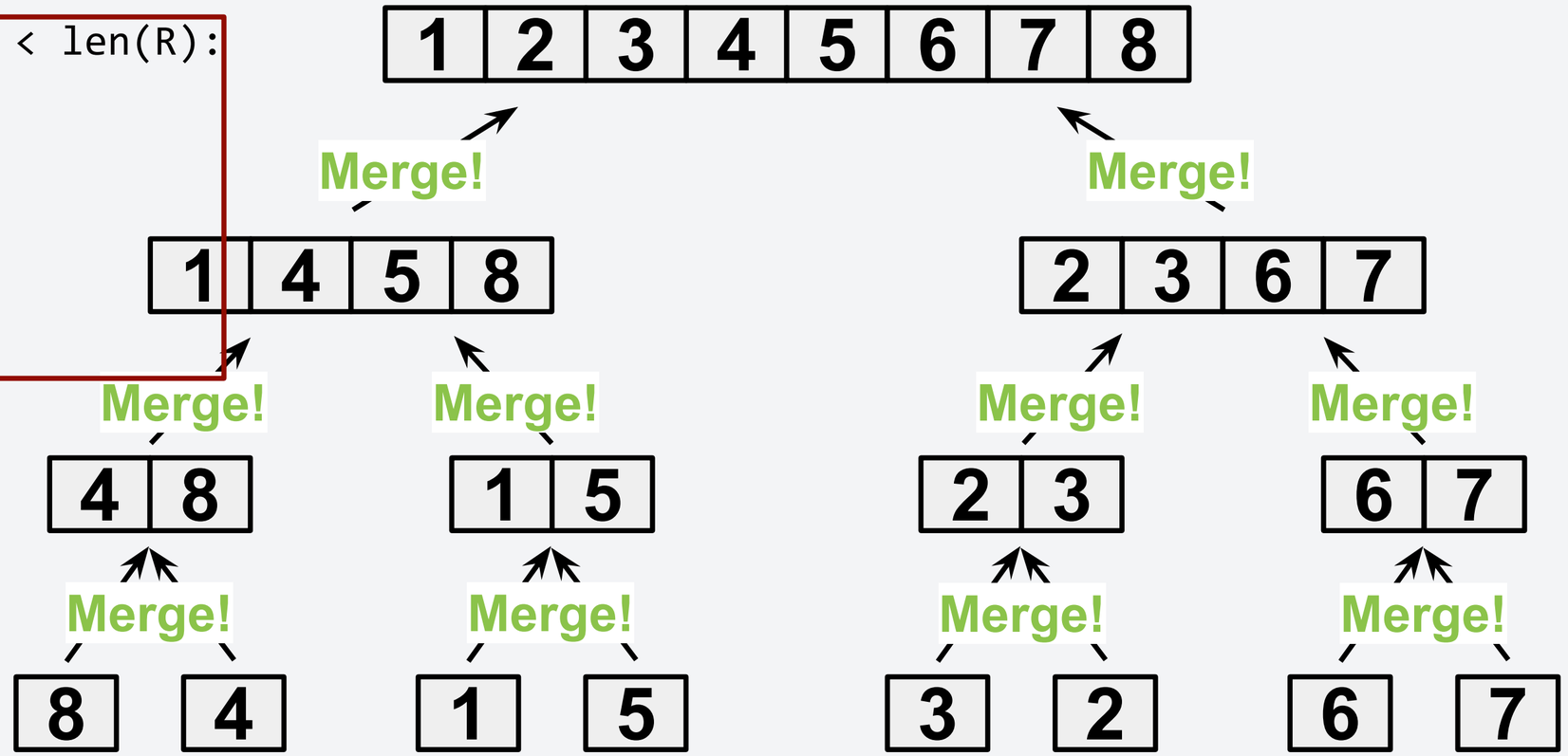
Sorting Algorithm 4: Merge Sort

Sample Python Implementation:

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

One we reach the **base case**, we merge the two sub-arrays.

Notice that when merging, we iterate over the elements of the sub-array once.



Demo: Merge Sort

Merge Sort - Proving its correctness

Consider first the correctness of **merge**() function:

Merge Sort - Proving its correctness

Consider first the correctness of **merge**() function:

Loop Invariant: At the start of iteration each iteration, `result` is a sorted list containing the elements from `L[0:l_idx]` and `R[0:r_idx]`

Merge Sort - Proving its correctness

Consider first the correctness of **merge**() function:

Loop Invariant: At the start of iteration each iteration, `result` is a sorted list containing the elements from `L[0:l_idx]` and `R[0:r_idx]`

Initialization: before first iteration, `result[]` is empty.

Merge Sort - Proving its correctness

Consider first the correctness of **merge**() function:

Loop Invariant: At the start of iteration each iteration, `result` is a sorted list containing the elements from `L[0:l_idx]` and `R[0:r_idx]`

Initialization: before first iteration, `result[]` is empty.

Maintenance: Assume loop invariant holds at the start of the i-th iteration. After i-th iteration, the following can happen:

```
R[r_idx] > L[l_idx] :   result.append(L[l_idx])(case1)

otherwise   :   result.append(R[r_idx])(case2)
```

- Since result[i-1] ≤ min(L[l_idx], R[r_idx]), appending either candidate, either case 1 or 2, will ensure that result[0:i] is still sorted.

Merge Sort - Proving its correctness

Consider first the correctness of **merge**() function:

Loop Invariant: At the start of iteration each iteration, `result` is a sorted list containing the elements from `L[0:l_idx]` and `R[0:r_idx]`

Initialization: before first iteration, `result[]` is empty.

Maintenance: Assume loop invariant holds at the start of the i-th iteration. After i-th iteration, the following can happen:

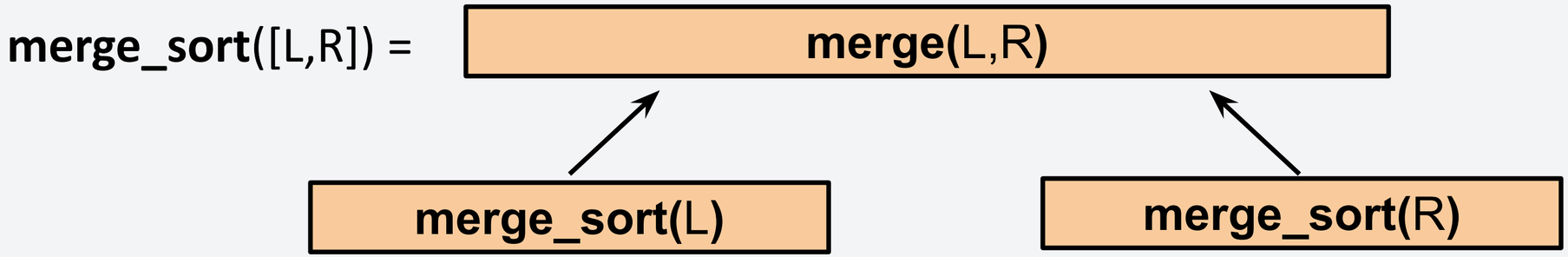
```
R[r_idx] > L[l_idx] :   result.append(L[l_idx])(case1)

otherwise   :   result.append(R[r_idx])(case2)
```

- Since result[i-1] ≤ min(L[l_idx], R[r_idx]), appending either candidate, either case 1 or 2, will ensure that result[0:i] is still sorted.

Termination: Loop terminates at when l_idx = len(L) or r_idx = len(R). At this point, result[] contains all elements of L[] and R[] (the remainder of the other list are appended.

Merge Sort - Proving its correctness

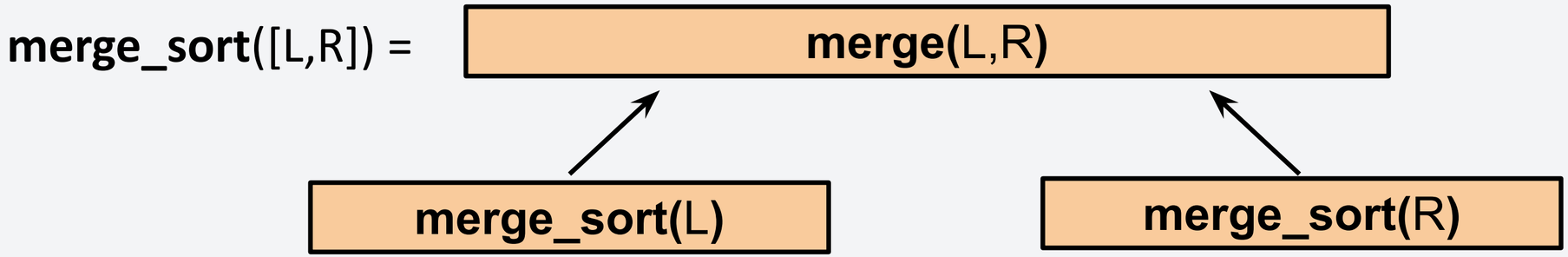
Then for **`merge_sort()`** we can use proof by (strong) induction:



Merge Sort - Proving its correctness

Then for **`merge_sort()`** we can use proof by (strong) induction:

Inductive Hypothesis: **`merge_sort()`** correctly sorts input lists of length i.

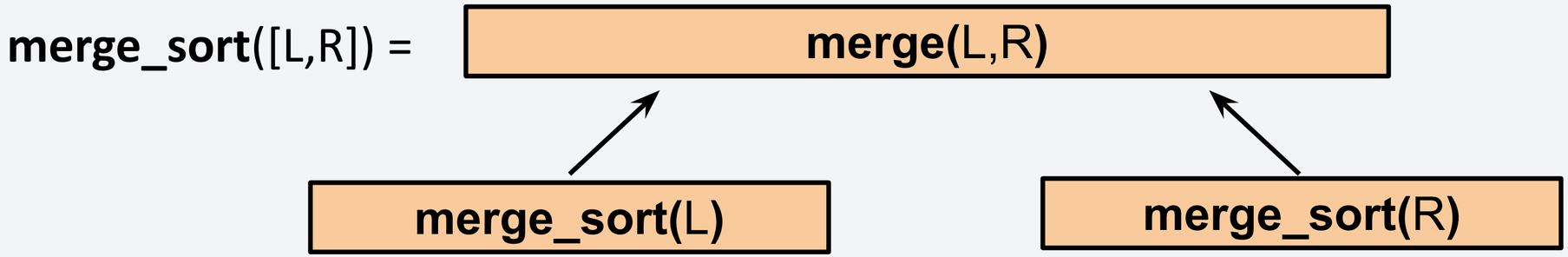


Merge Sort - Proving its correctness

Then for **`merge_sort()`** we can use proof by (strong) induction:

Inductive Hypothesis: **`merge_sort()`** correctly sorts input lists of length i.

Base Case: **`merge_sort()`** correctly sorts input lists of length 1. It returns a 1-element list. Thus, it is trivially sorted.



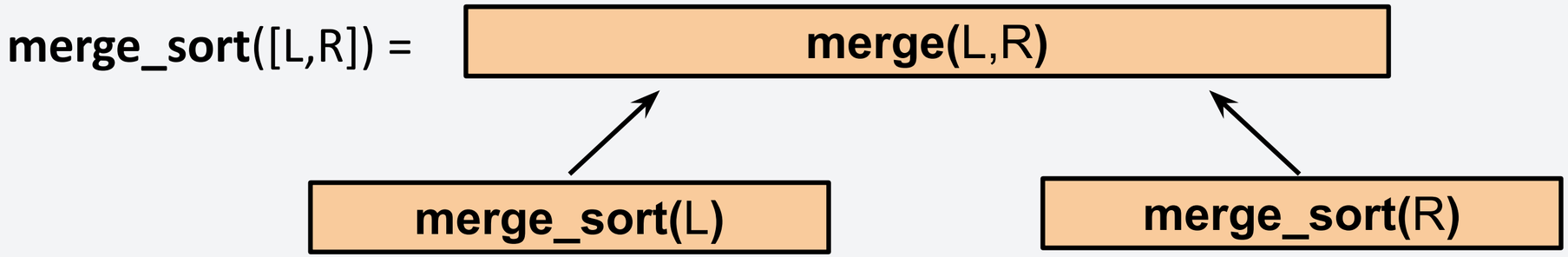
Merge Sort - Proving its correctness

Then for **`merge_sort()`** we can use proof by (strong) induction:

Inductive Hypothesis: **`merge_sort()`** correctly sorts input lists of length i.

Base Case: **`merge_sort()`** correctly sorts input lists of length 1. It returns a 1-element list. Thus, it is trivially sorted.

Inductive Step: Given two sorted lists, **`merge()`** returns a single sorted list with all the elements from the original two lists. This is proven previously.



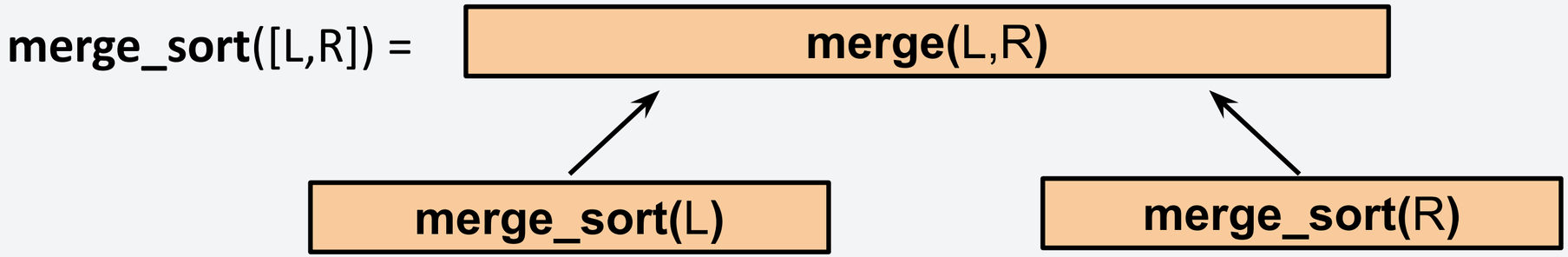
Merge Sort - Proving its correctness

Then for **`merge_sort()`** we can use proof by (strong) induction:

Inductive Hypothesis: **`merge_sort()`** correctly sorts input lists of length i.

Base Case: **`merge_sort()`** correctly sorts input lists of length 1. It returns a 1-element list. Thus, it is trivially sorted.

Inductive Step: Given two sorted lists, **`merge()`** returns a single sorted list with all the elements from the original two lists. This is proven previously.



Since merge_sort(i) is assumed to be true, the L and R lists are already sorted.

And since merge() is proven, the inductive step is also true.

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).
- Calling merge_sort() on list of length n calls merge_sort() once for each half.

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).
- Calling merge_sort() on list of length n calls merge_sort() once for each half.
- Question is, what is the runtime of merge_sort()? Let’s check the recurrence relation.

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).
- Calling merge_sort() on list of length n calls merge_sort() once for each half.
- Question is, what is the runtime of merge_sort()? Let's check the recurrence relation.

From our python implementation,

```
def merge_sort(A):          #T(n)
    if len(A) <= 1:
      return A              #c0
    L = merge_sort(A[0:n/2])      #T(⌈ n/2 ⌉) + c
    R = merge_sort(A[n/2:n])      #T(⌊ n/2 ⌋) + c
    return merge(L, R)      #complexity of merge()?
```

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).
- Calling merge_sort() on list of length n calls merge_sort() once for each half.
- Question is, what is the runtime of merge_sort()? Let's check the recurrence relation.

From our python implementation,

```
def merge_sort(A):          #T(n)
    if len(A) <= 1:
      return A              #c0
    L = merge_sort(A[0:n/2])      #T(⌈ n/2 ⌉) + c
    R = merge_sort(A[n/2:n])      #T(⌊ n/2 ⌋) + c
    return merge(L, R)      #complexity of merge()?
```

$$T_{merge\_sort}(n) = T_{merge\_sort}(\lfloor n/2 \rfloor) + T_{merge\_sort}(\lceil n/2 \rceil) + c_1 + T_{merge}(n)$$

Merge Sort - Runtime Analysis

Let T(n) be the runtime of merge_sort() on a list of length n.

- For a list of length n/2, the runtime of merge_sort() is T(n/2).
- Calling merge_sort() on list of length n calls merge_sort() once for each half.
- Question is, what is the runtime of merge_sort()? Let's check the recurrence relation.

From our python implementation,

```
def merge_sort(A):                  #T(n)
    if len(A) <= 1:
        return A                    #c0
    L = merge_sort(A[0:n/2])        #T(⌈ n/2 ⌉) + c
    R = merge_sort(A[n/2:n])        #T(⌊ n/2 ⌋) + c
    return merge(L, R)              #complexity of merge()?
```

$$T_{merge\_sort}(n) = T_{merge\_sort}(\lfloor n/2 \rfloor) + T_{merge\_sort}(\lceil n/2 \rceil) + c_1 + T_{merge}(n)$$

What's the time complexity of merge()?

Merge Sort - Runtime Analysis

From our implementation of merge(),

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

Merge Sort - Runtime Analysis

From our implementation of merge(),

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

At most, this runs for len(L) + len(R) which is n

$$T_{merge}(n) = O(n)$$

Merge Sort - Runtime Analysis

From our implementation of merge(),

```
def merge(L, R):
  result = []
  l_idx, r_idx = (0, 0)
  while l_idx < len(L) and r_idx < len(R):
    if L[l_idx] < R[r_idx]:
      result.append(L[l_idx])
      l_idx += 1
    else:
      result.append(R[r_idx])
      r_idx += 1
  result.extend(L[l_idx:len(L)])
  result.extend(R[r_idx:len(R)])
  return result
```

At most, this runs for len(L) + len(R) which is n

$$T_{merge}(n) = O(n)$$

Thus, the final recurrence relation is:

$$T_{m\_sort}(n) = \begin{cases} T_{m\_sort}(\lceil n/2 \rceil) + T_{m\_sort}(\lceil n/2 \rceil) + c_1 n, & n > 1 \\ c_0, & n = 1 \end{cases}$$

Merge Sort - Runtime Analysis

To solve for the runtime, we do the substitution method. Evaluating a few steps,

$$
\begin{aligned}
T_{m\_sort}(n) &= 2 \cdot T_{mx\_sort}\left({}^{n}/_{2}\right) + c_1 n \\
&= 4 \cdot T_{m\_sort}\left({}^{n}/_{4}\right) + c_1 n + c_1 n \\
&= 8 \cdot T_{m\_sort}\left({}^{n}/_{8}\right) + c_1 n + c_1 n + c_1 n \\
&\vdots \\
&= 2^k \cdot T_{m\_sort}\left({}^{n}/_{2^k}\right) + c_1 n k
\end{aligned}
$$

Setting $n = 2^k$ to reach the base case,

$$
T_{m\_sort}(n) = \underbrace{2^{\log_2 n}}_{\text{simplifies to n}} \cdot T_{m\_sort}(1) + c_1 \cdot n \log_2 n = O(n \cdot log_2 n)
$$

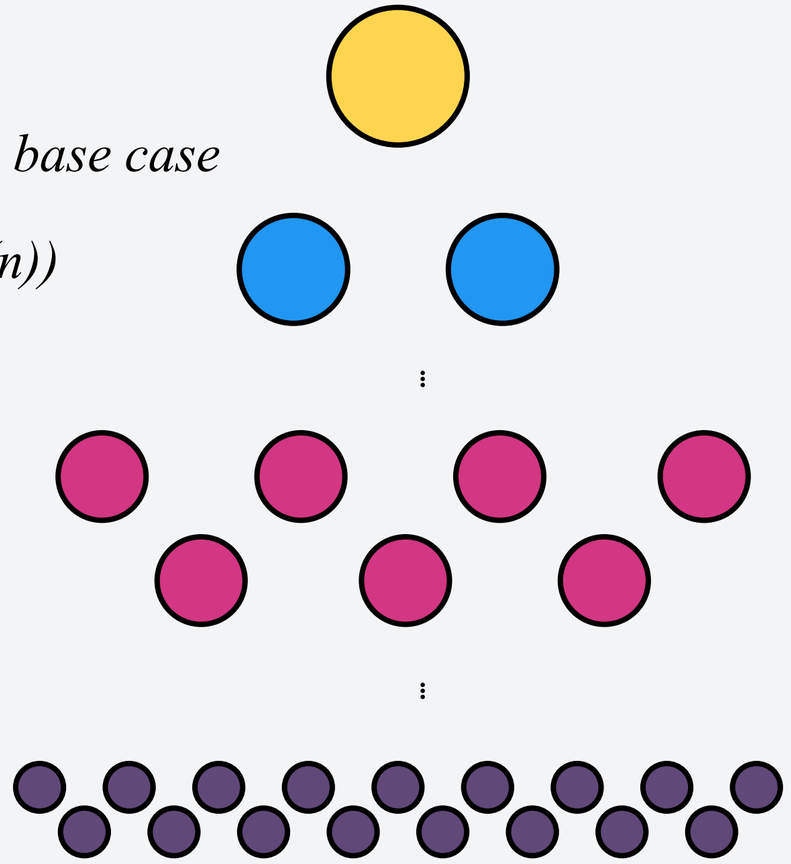
Merge Sort - Runtime Analysis

Not convinced? Let's look at it visually,

Runtime

$$T(n) = (work\ per\ level)(num\ of\ levels) + base\ case$$

$$T(n) = cnlog_2(n) + cn = O(nlog(n))$$



1 problem of size cn — **cn**

2 problems of size cn/2 — **cn**

2^t problems of size $cn/2^t$ — **cn**

$2^{log\_2(n)} = n$ problems of size c — **cn**

$log_2 n$ levels

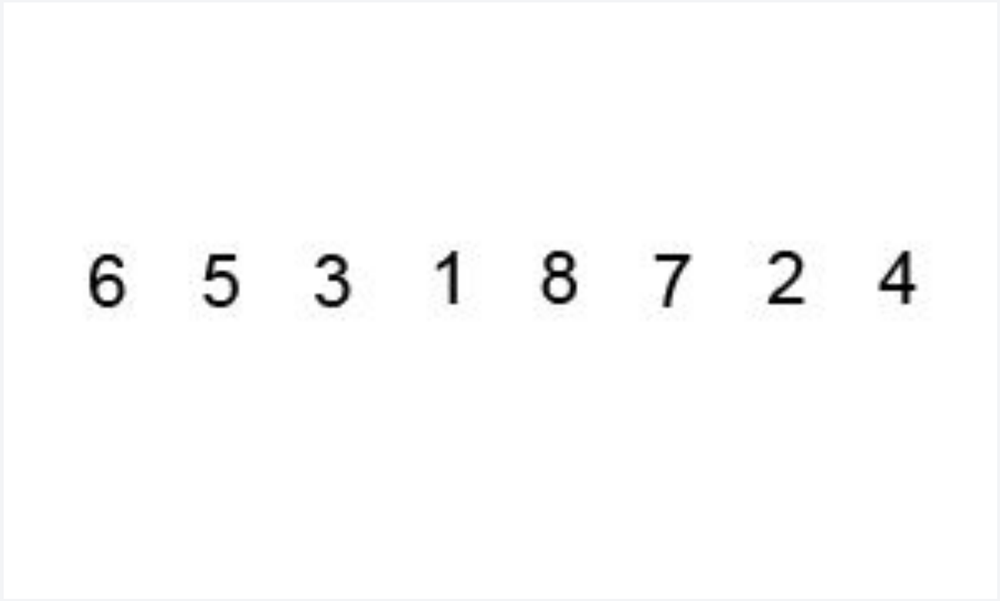
Let’s continue with the sorting algorithms

Let’s practice more divide-and-conquer algorithms by continuing with our sorting algorithms. There are two more sorting algorithms we are going to explore:

Algorithm	Proved?	Time Complexity	
		Best Case	Worst Case
Insertion Sort	Yes!	O(n)	$O(n^2)$
Bubble Sort	Yes!	O(n)	$O(n^2)$
Selection Sort	Assignment :)	$O(n^2)$	$O(n^2)$
Merge Sort	Yes!	O(nlogn)	O(nlogn)
Quick Sort			

Sorting Algorithm 5: Quicksort

Quicksort works with the concept called **pivots**. Essentially, we set a pivot from the array (usually the last element), then partition the array such that the elements at the left of the pivot is less than the pivot, then greater than at the right of the pivot.



Pseudocode:

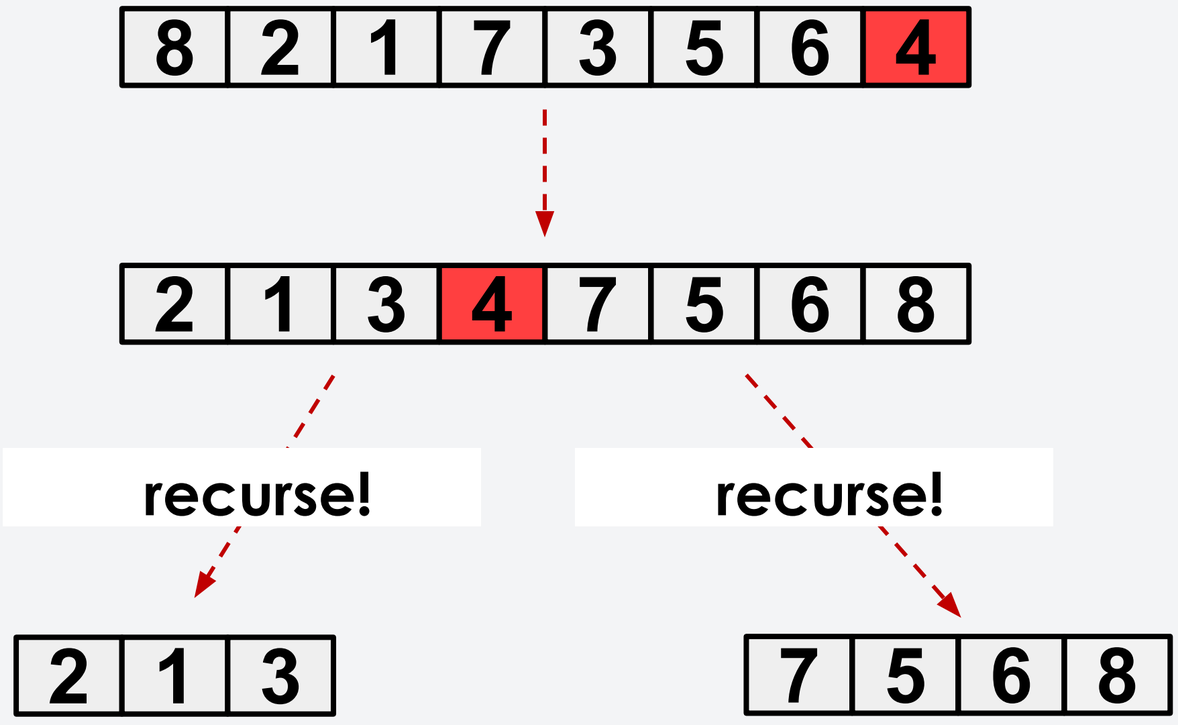
```
for each (unsorted) partition
set last element as pivot
  storeIndex = leftmostIndex
  for i = leftmostIndex to pivotIndex-1
    if ((a[i] < a[pivot]) or (equal but 50% lucky))
      swap(i, storeIndex); ++storeIndex;
  swap(pivot, storeIndex)
```

We don’t need to fully sort the left and right side of the array. We just have to ensure that the values at the left and right of the pivot are less than and greater than the pivot, respectively.

Sorting Algorithm 5: Quicksort

Quicksort is typically implemented using two functions:

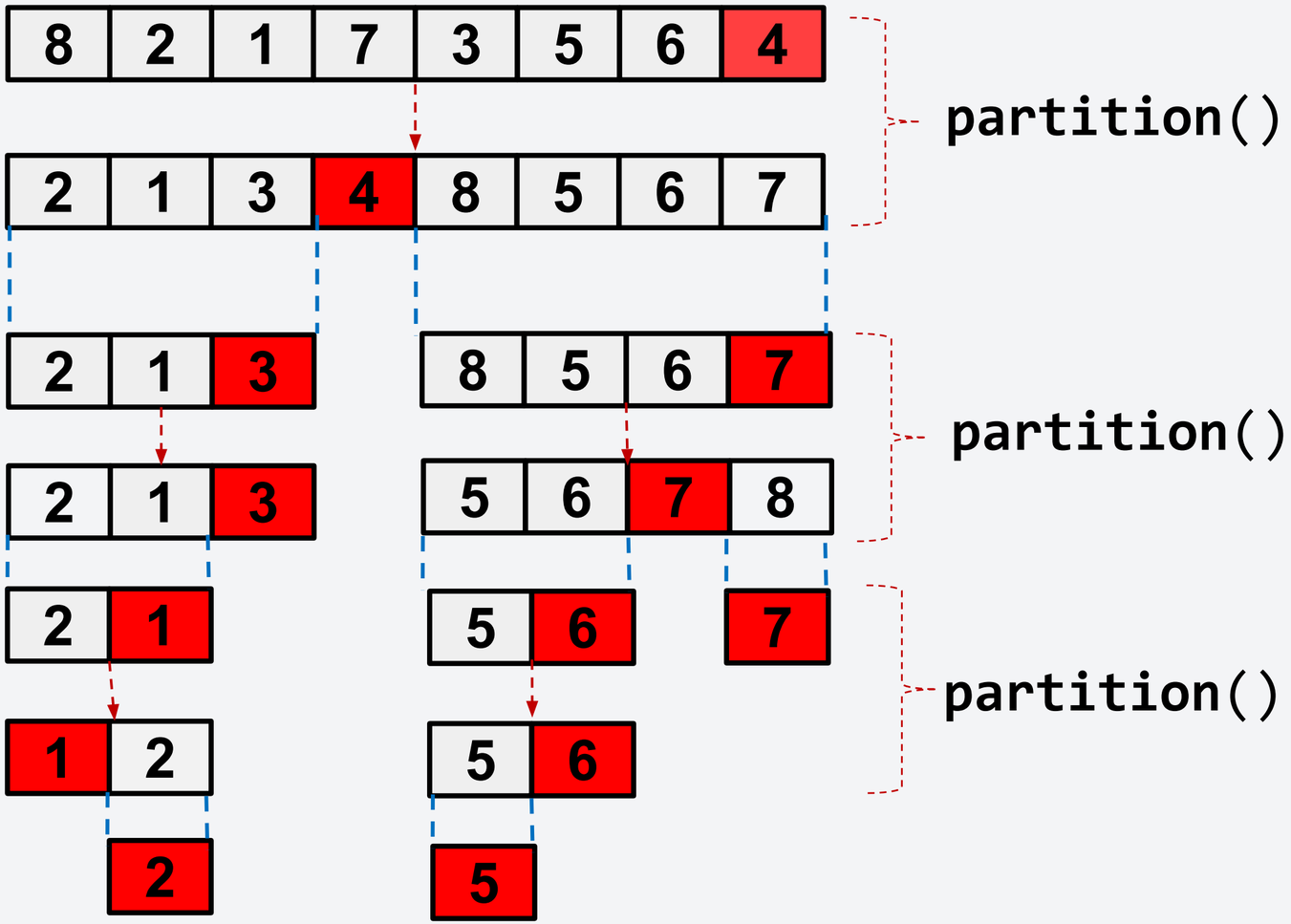
1. partition()
   a. Pick an element as pivot, usually the last.
   b. Rearrange the order such that the elements on the left of the pivot are less while the elements on the right are greater than the pivot.
   c. Return the position of the pivot.
2. quick_sort()
   a. Recursively apply partition() on the left and right of the pivot.



How did we move the pivot? Follow the pseudocode!

Sorting Algorithm 5: Quicksort

Full example:



Sorting Algorithm 5: Quicksort

Full example:

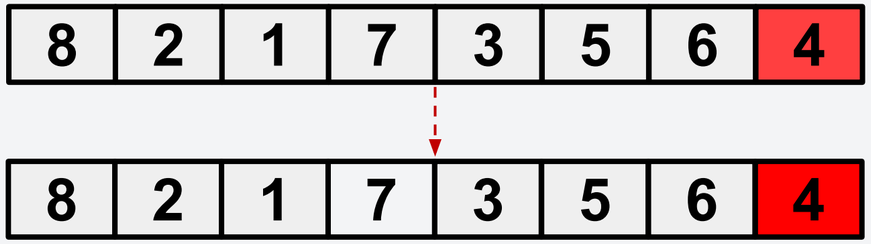
Let's try the first iteration.

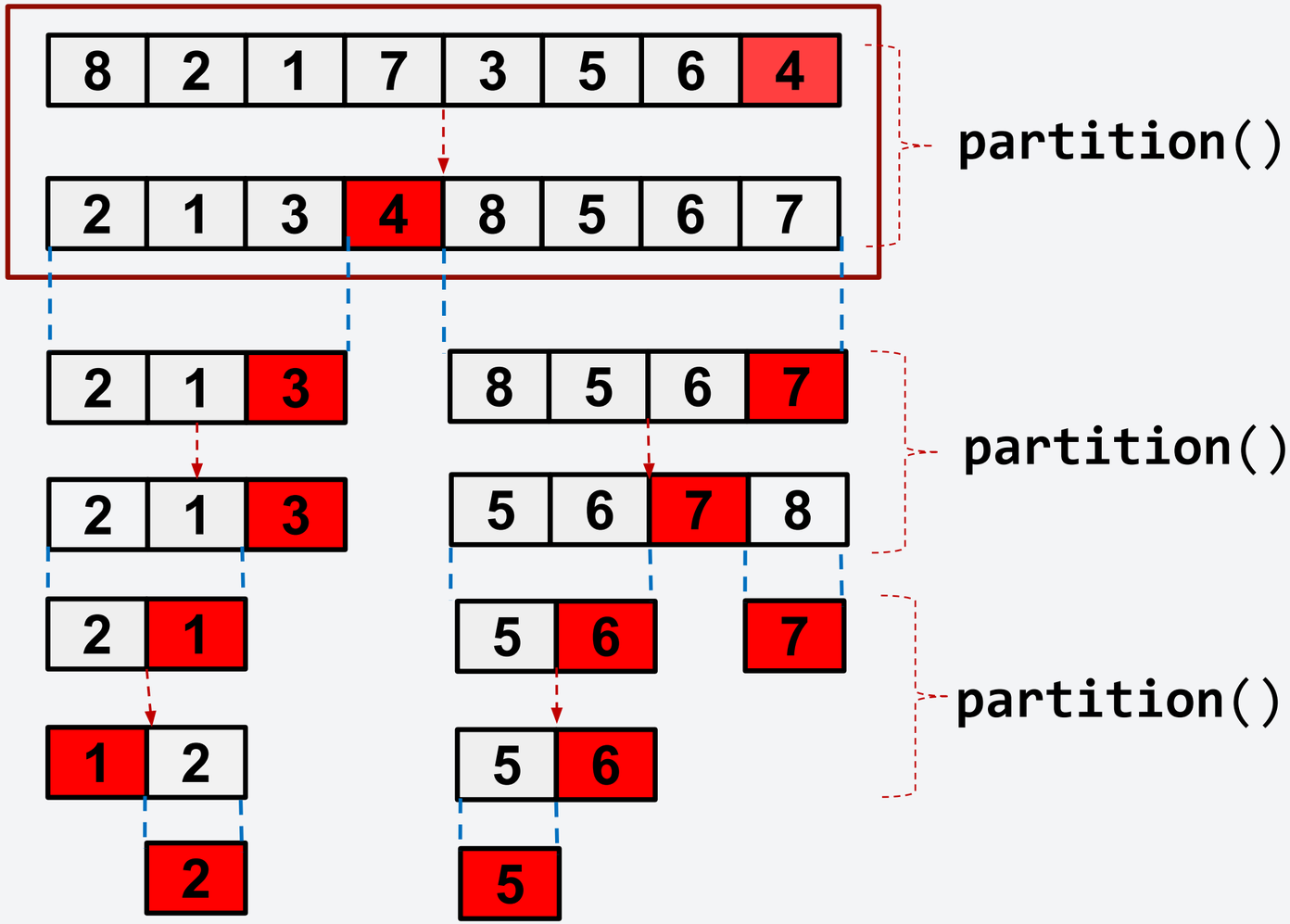
storeIndex = 0

pivotIndex = 7

i = 0

a[0] < a[pivotIndex]?
False, no swap.





Sorting Algorithm 5: Quicksort

Full example:

Let’s try the first iteration.

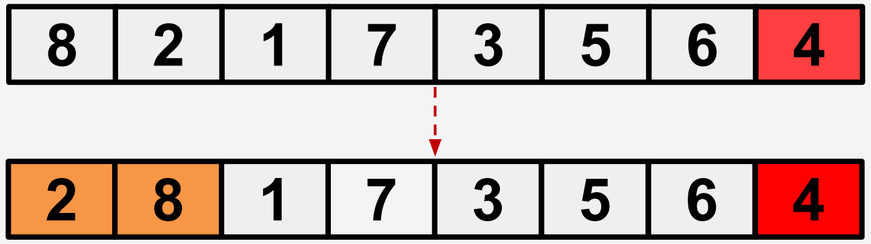
storeIndex = 0 -> 1

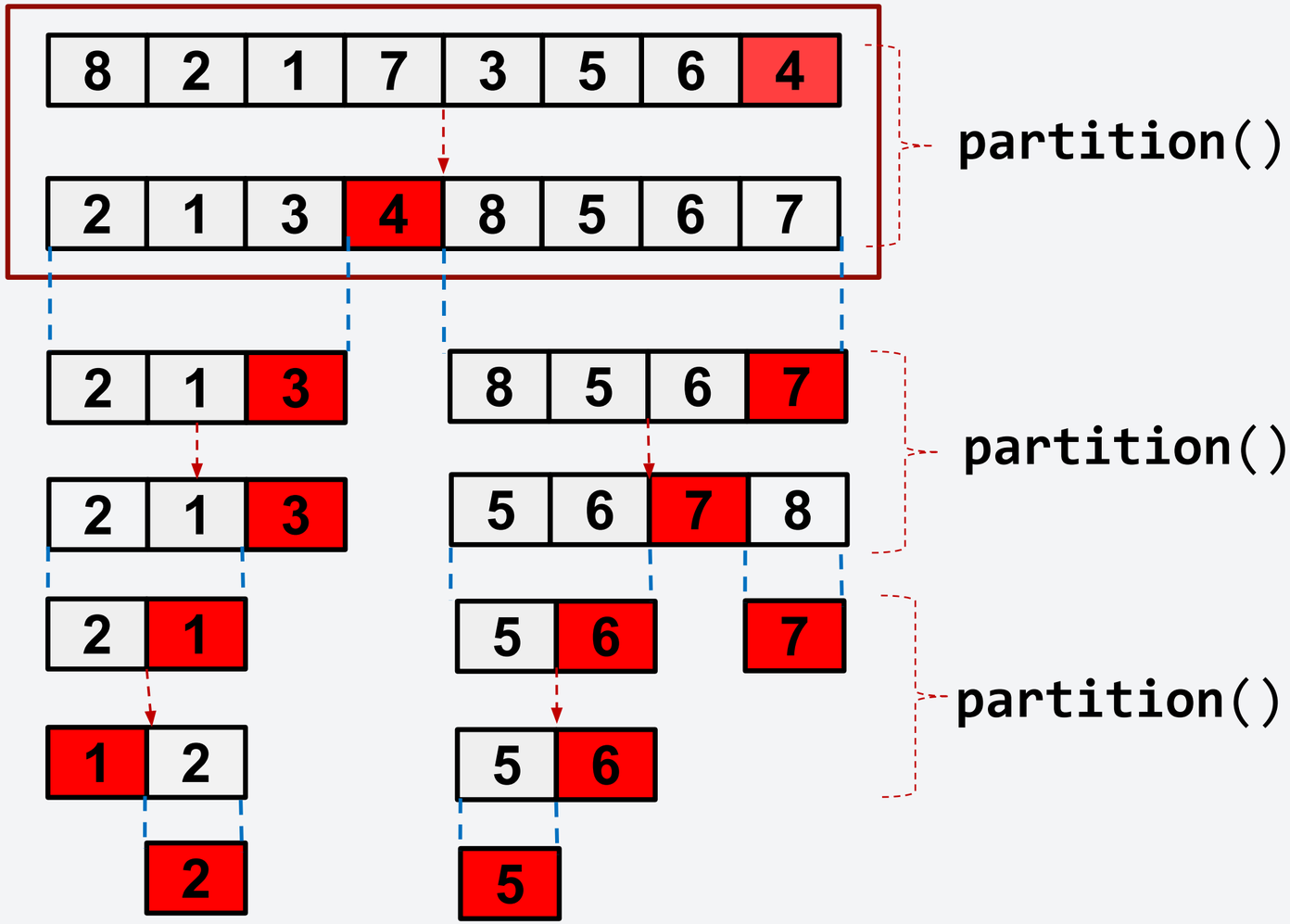
pivotIndex = 7

i = 1

a[1] < a[pivotIndex]?

True, swap a[storeIndex] and a[i], add +1 to storeIndex.





Sorting Algorithm 5: Quicksort

Full example:

Let’s try the first iteration.

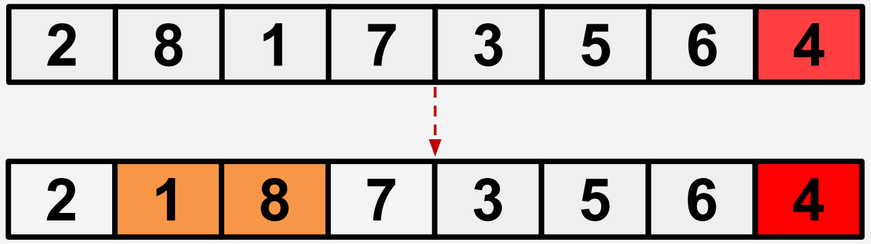
storeIndex = 1 -> 2

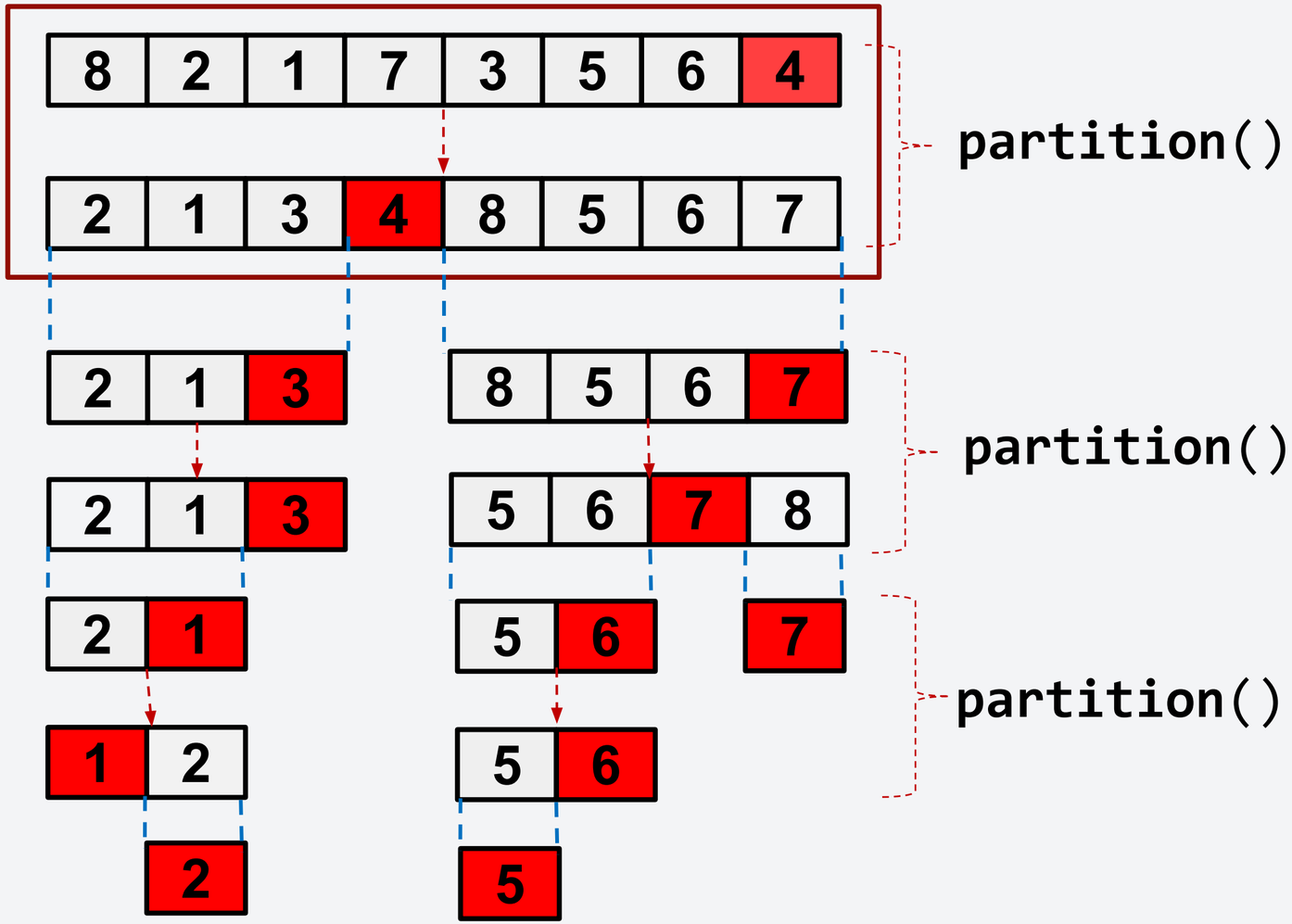
pivotIndex = 7

i = 2

a[2] < a[pivotIndex]?

True, swap a[storeIndex] and a[2], add +1 to storeIndex.





Sorting Algorithm 5: Quicksort

Full example:

Let’s try the first iteration.

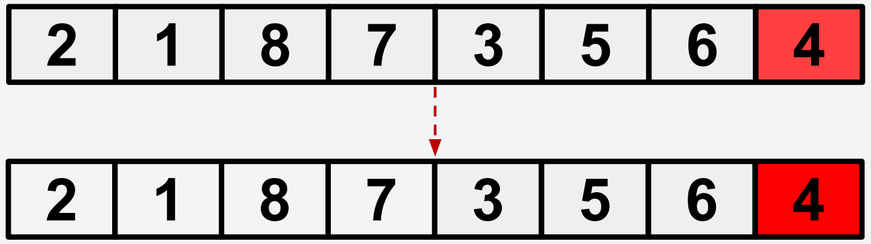
storeIndex = 2

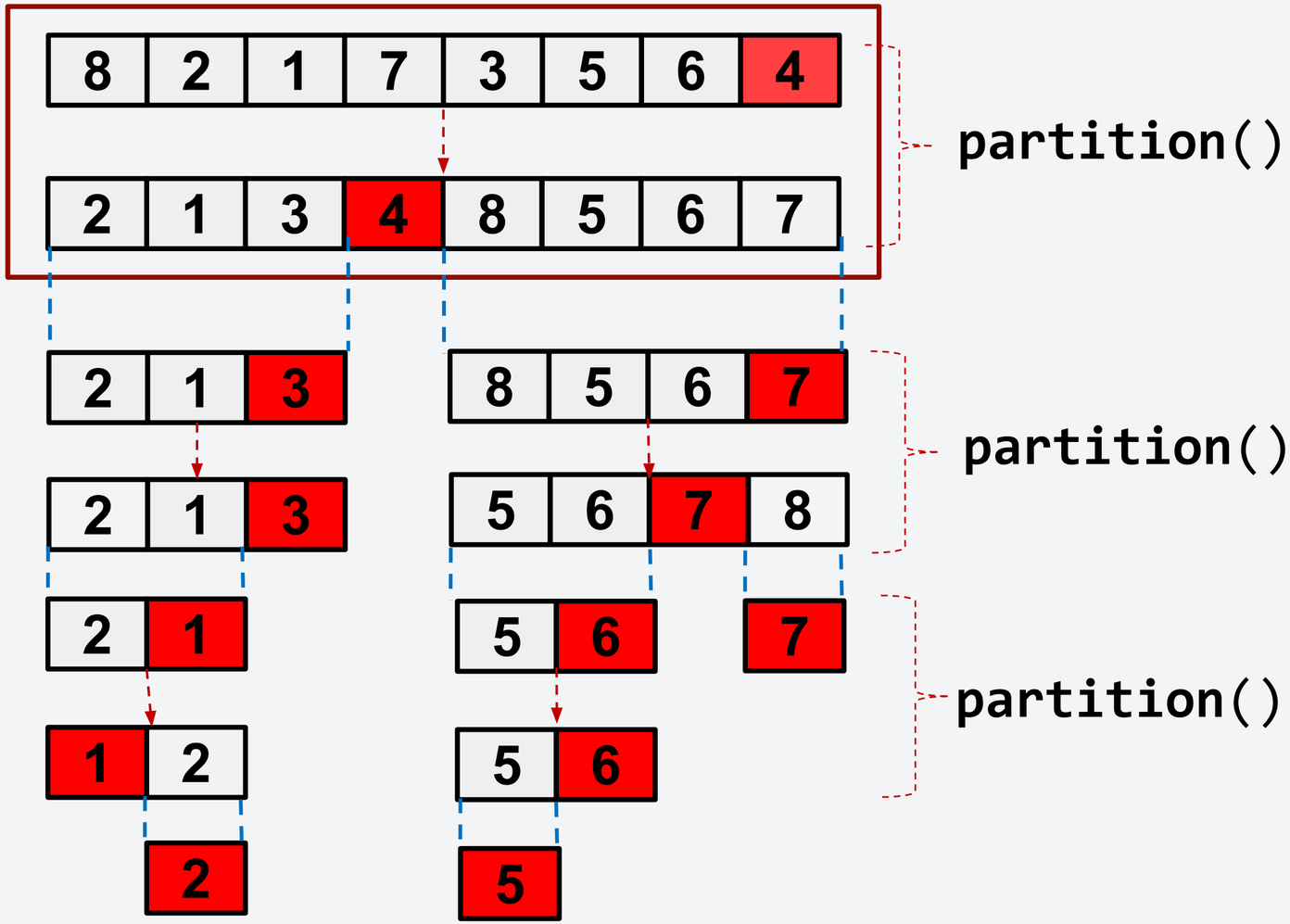
pivotIndex = 7

i = 3

a[3] < a[pivotIndex]?

False, do nothing.





Sorting Algorithm 5: Quicksort

Full example:

Let’s try the first iteration.

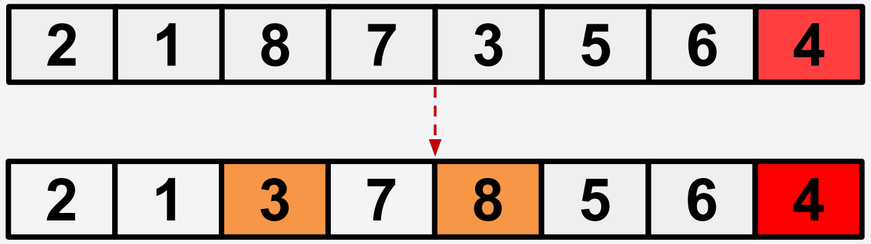
storeIndex = 2 **-> 3**

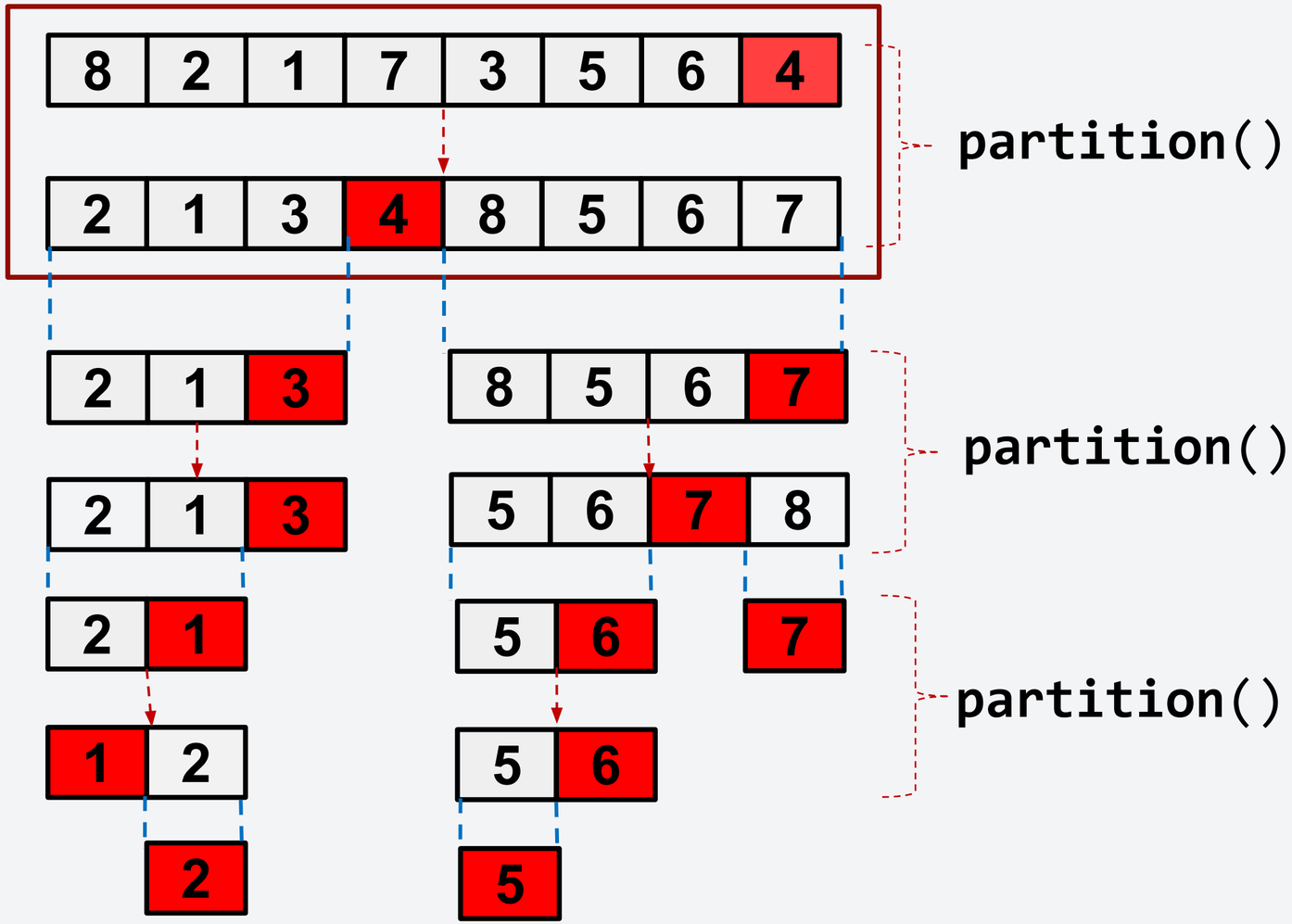
pivotIndex = 7

i = 4

a[4] < a[pivotIndex]?

True, swap a[storeIndex] and a[4], add +1 to storeIndex.





Sorting Algorithm 5: Quicksort

Full example:

Let’s try the first iteration.

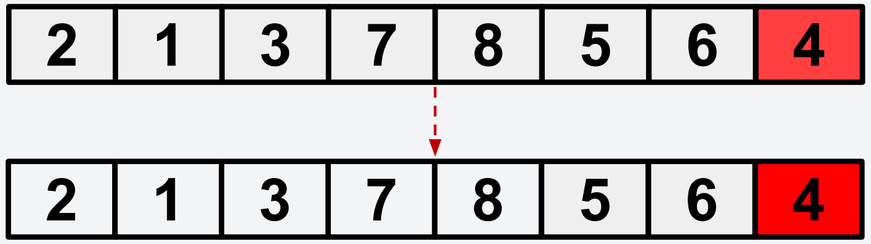
storeIndex = 3

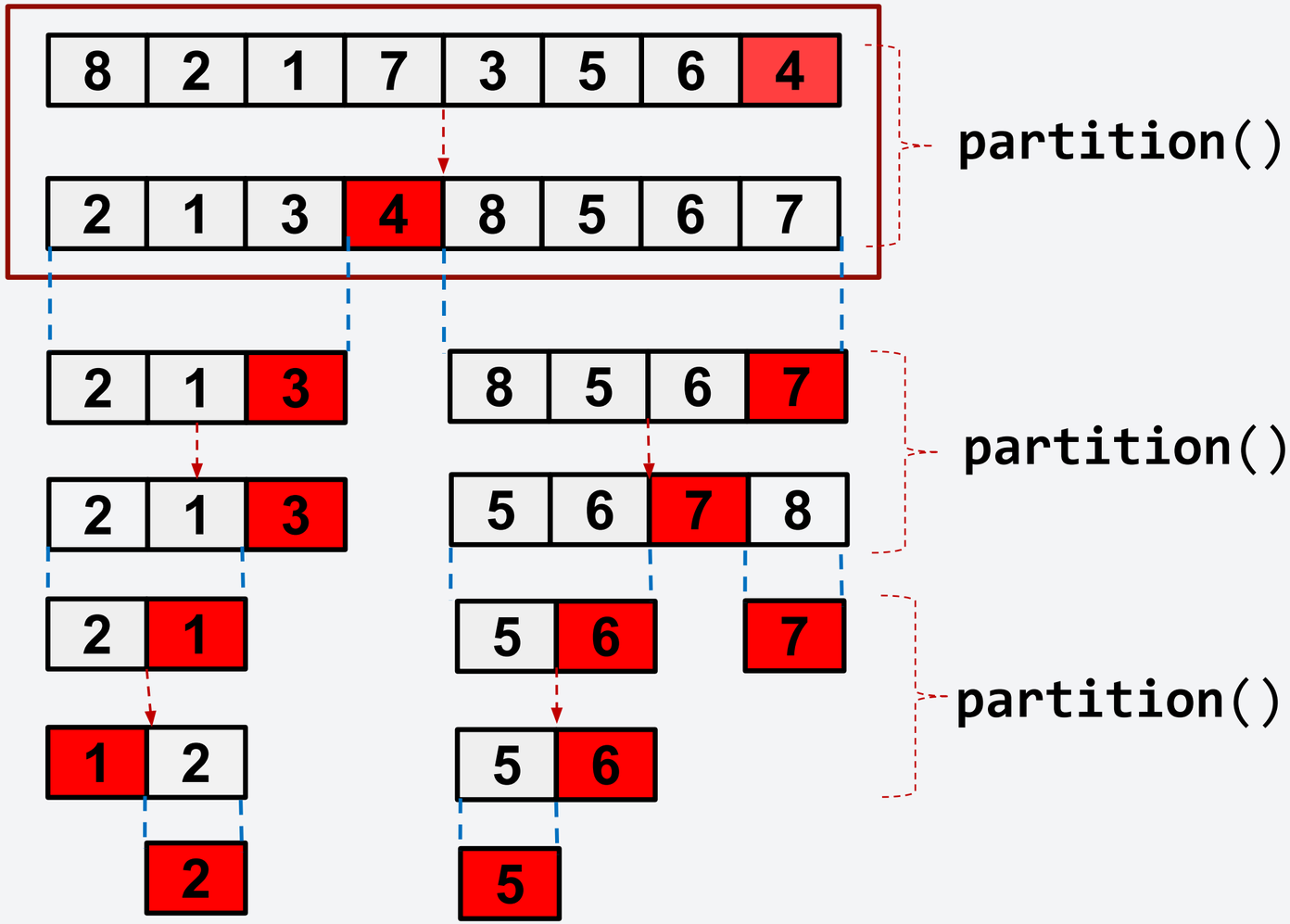
pivotIndex = 7

i = 5,6

a[i] < a[pivotIndex]?

False, do nothing.





Sorting Algorithm 5: Quicksort

Full example:

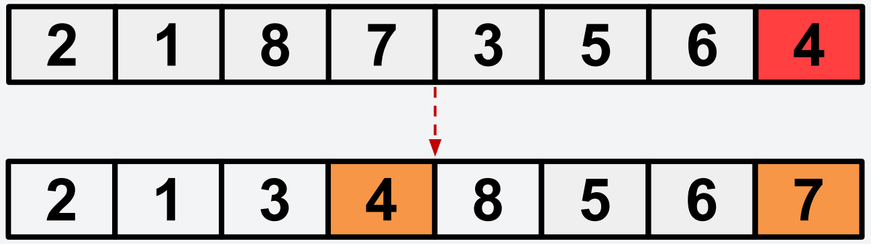
Let’s try the first iteration.

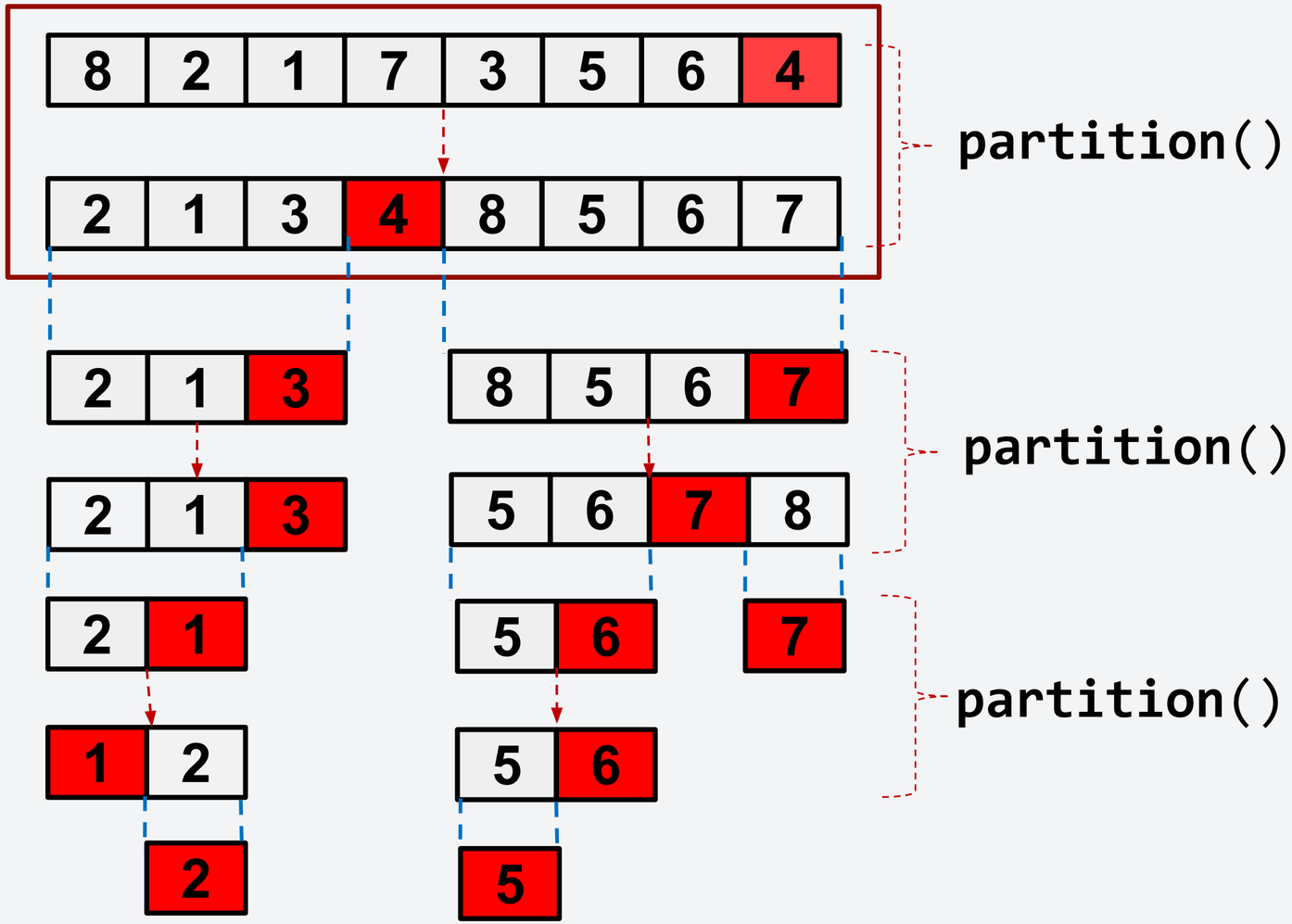
storeIndex = 2 **-> 3**

pivotIndex = 7

i = 5,6

After the loop, swap storeIndex with pivotIndex.





Sorting Algorithm 5: Quicksort

Full example:

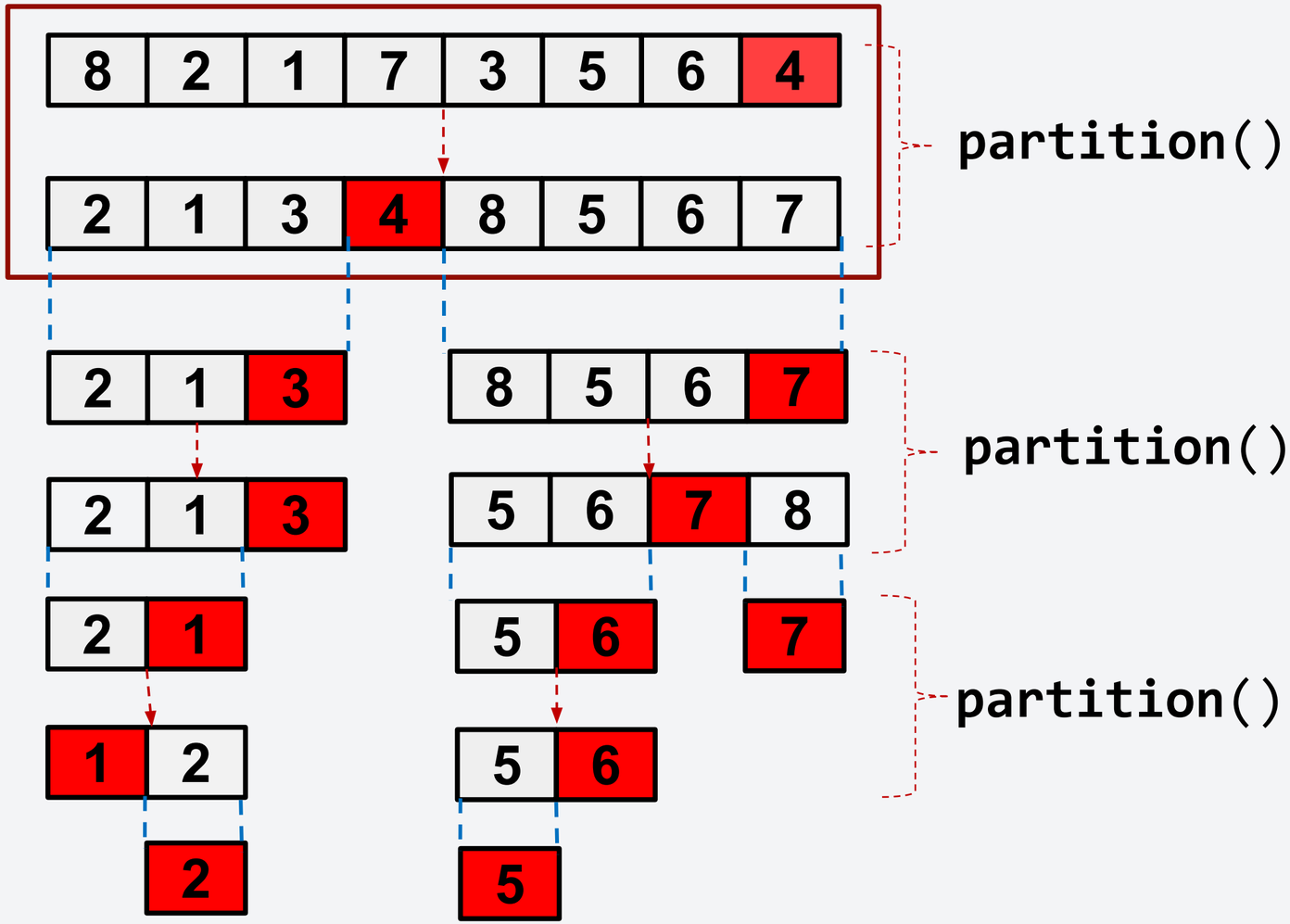
Let’s try the first iteration.

storeIndex = 2 **-> 3**

pivotIndex = 7

i = 5,6

After the loop, swap storeIndex with pivotIndex.



Sorting Algorithm 5: Quick Sort

Sample C++ Implementation:

```
void quicksort(int* arr,int lo, int hi) {
    if(lo < hi - 1) {
        int p = partition(arr,lo,hi);
        quicksort(arr,lo,p);
        quicksort(arr,p+1,hi);
    }
}
```

```
int partition(int* arr,int lo, int hi) {
    int pivot = arr[hi - 1];
    int i = lo;
    int j = hi - 2;
    while(i <= j) {
        if(arr[i] > pivot && arr[j] <
pivot) {
            swap(arr[i],arr[j]);
            i++;
            j--;
        } else if(arr[i] <= pivot) {
            i++;
        } else {
            j--;
        }
    }
    if(arr[i] > arr[hi-1])
        swap(arr[i],arr[hi - 1]);
    return i;
}
```

Quick Sort - Analysis

HINTS on proving correctness:

1. Identify a loop invariant in the partition() function and use this to prove the correctness of partition() function
2. Apply induction on quick_sort() function. This requires correctness of partition() function (which you should have proven in step 1)

HINTS on analyzing runtime:

- Important assumption: partition function always divides the list almost equally
- Picking pivots at random (why?) Randomized Quick Sort
- Recurrence relation of (Randomized) Quick Sort will have the same form as that of Merge Sort
- Runtime: O(nlog2n)

Quick Sort - Analysis

HINTS on proving correctness:

1. Identify a loop invariant in the partition() function and use this to prove the correctness of partition() function
2. Apply induction on quick_sort() function. This requires correctness of partition() function (which you should have proven in step 1)

HINTS on analyzing runtime:

- Important assumption: partition function always divides the list almost equally
- Picking pivots at random (why?) Randomized Quick Sort
- Recurrence relation of (Randomized) Quick Sort will have the same form as that of Merge Sort
- **Runtime: $O(n\log_2(n))$**

This will be your assignment :D

Deadline: March 11, 2025 11:59PM

- Recursion Overview
- Divide-and-Conquer Algorithms
- **Recursive Backtracking**

Lumingon ka sa pinaggalingan mo

Recursive Backtracking: using recursion to explore solutions to a problem and abandoning them if they are not suitable.

- Determine whether a solution exists
- Find the best solution
- Count the number of solutions
- Print/find all the solutions

Applications:

- Puzzle solving (sudoku, crossword puzzles, etc.)
- Game playing (chess, solitaire, etc.)
- Constraint satisfaction problems (scheduling, matching, etc.)

How to implement backtracking solutions?

As mentioned, backtracking is used when we want to explore **all possible** solutions given a certain goal together with the possibility of having some constraints.

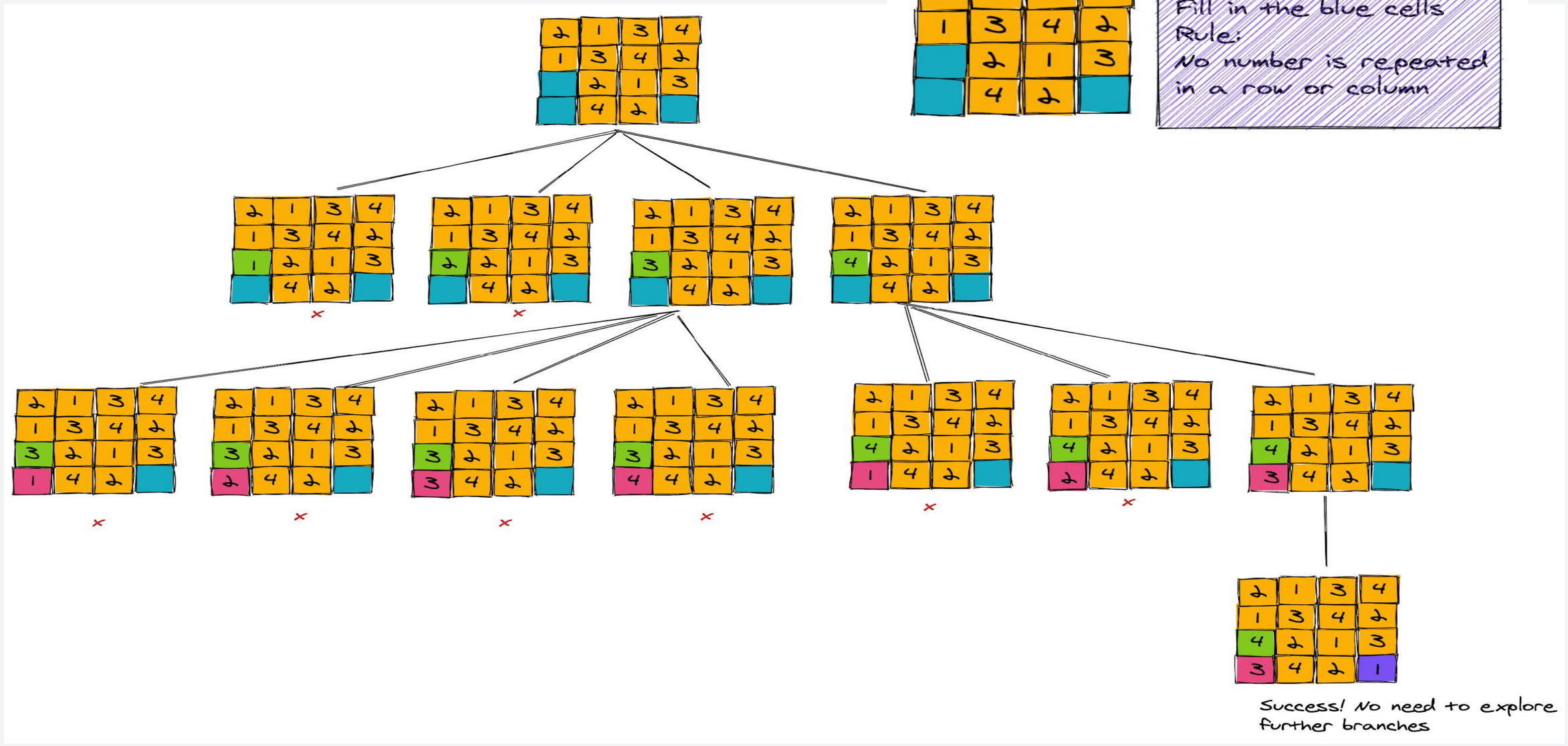
How to implement backtracking solutions?

As mentioned, backtracking is used when we want to explore **all possible** solutions given a certain goal together with the possibility of having some constraints.

Recursive backtracking is kinda similar to **trial-and-error**:

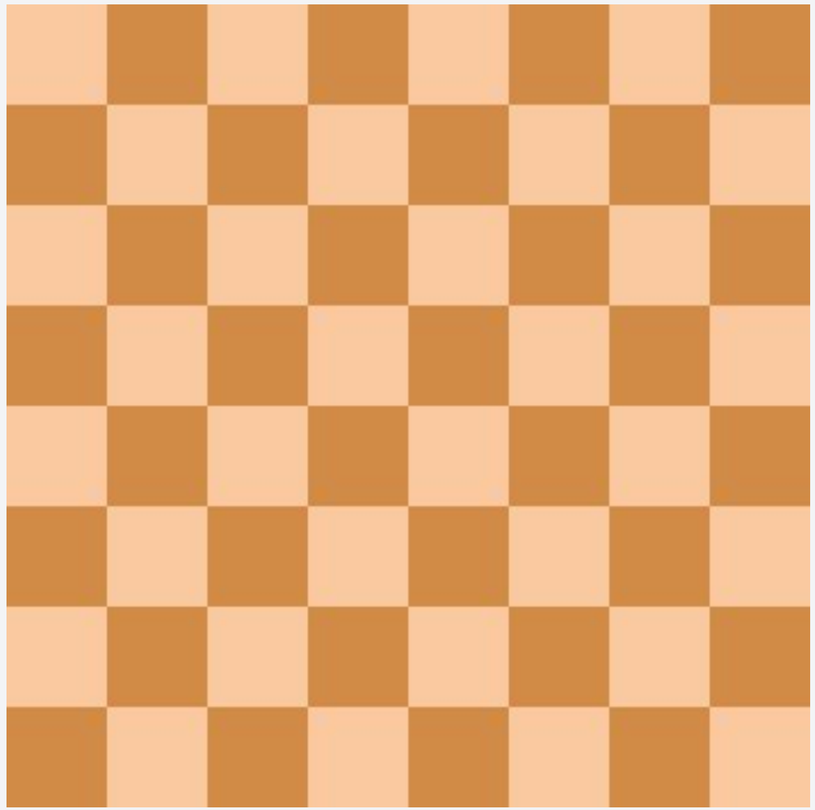
1. Given your current position, make a choice.
    a. Choose the next most appropriate step given all possible options.
2. Explore the next step using **recursion**.
    a. Speed up the exploration by using recursion.
3. Make sure to have the ability to undo the choice (**backtracking**).
    a. If the current path does not provide a solution, revert to the last choice and try a different path.

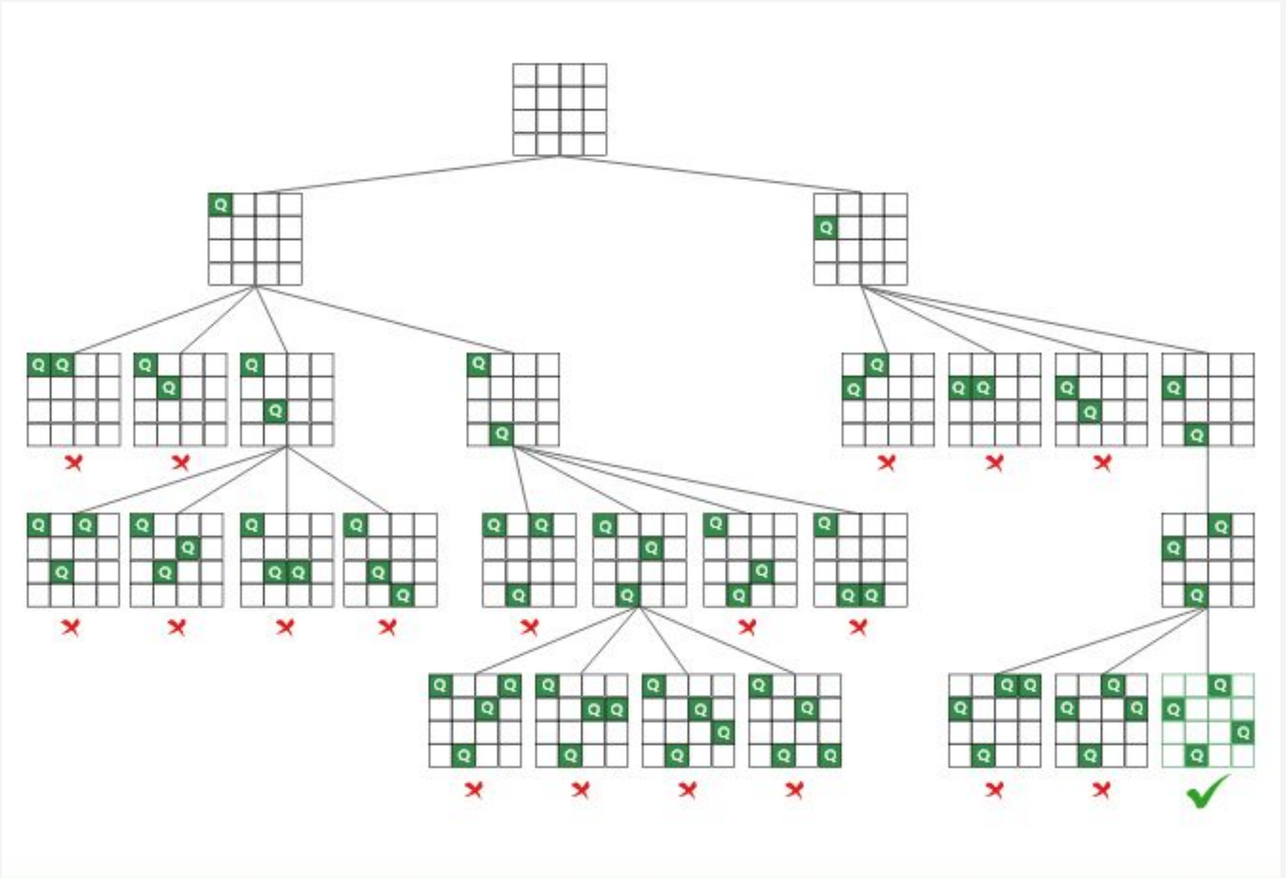
Example: Sudoku



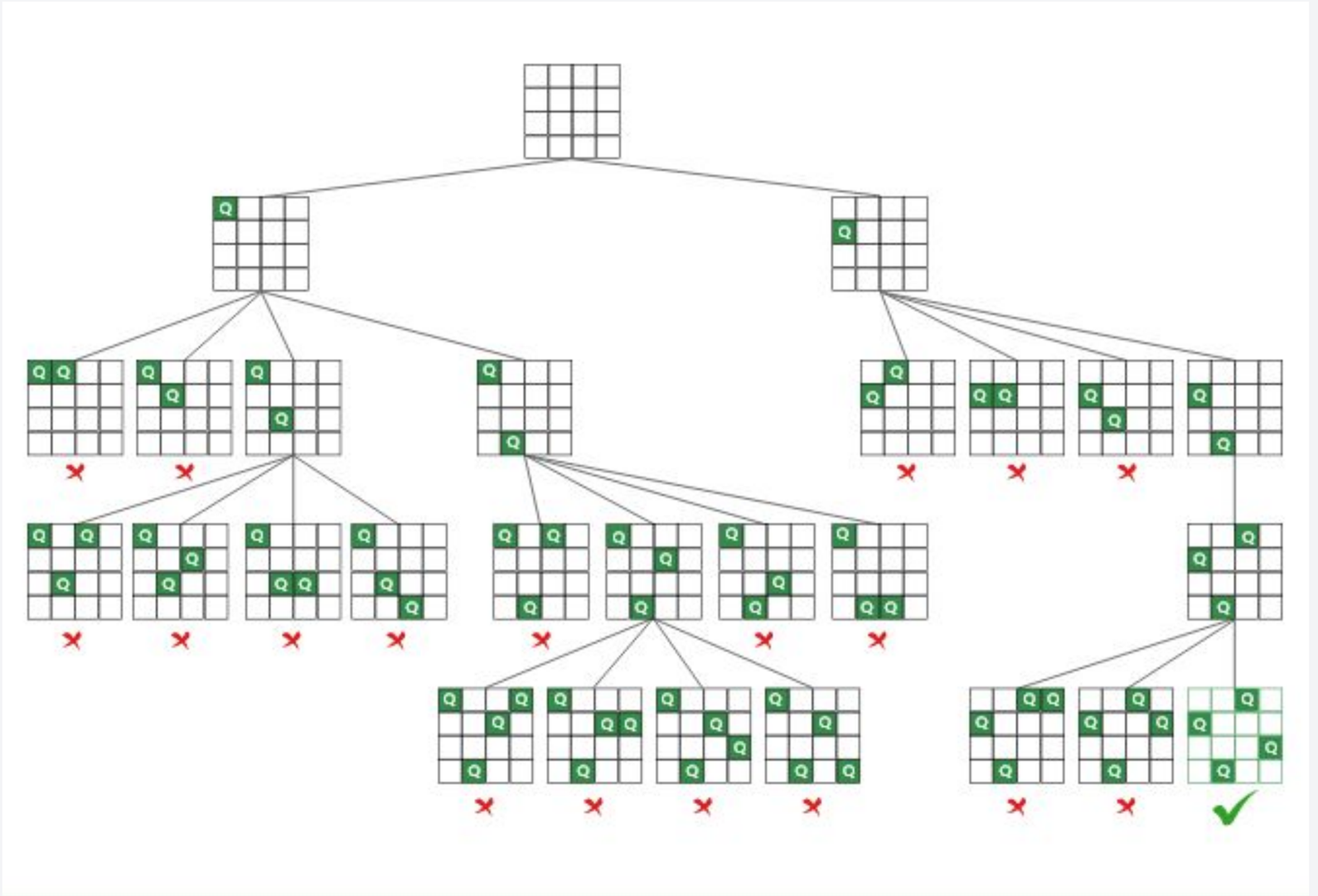
Example: N-queens problem

Task: In an nxn chessboard, place N queens such that the queens can’t attack each other.





Example: N-queens problem



Practice: Printing binary numbers

Suppose we want to create a recursive function **`printAllBinary()`** that accepts an integer input n representing the number of binary digits. Print the binary digits per line and in **ascending order**.

- What is the base case?
- What should be the recursive call?
- How do we ensure that we print in ascending order?

```
- printAllBinary(2);
  00
  01
  10
  11
```

```
printAllBinary(3);
000
001
010
011
100
101
110
111
```

Practice: Printing binary numbers

Sample Implementation:

```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
  printAllBinaryHelper(digits – 1,soFar + “0”);
  printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}


void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

Practice: Printing binary numbers

Sample Implementation:

```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
  printAllBinaryHelper(digits – 1,soFar + “0”);
  printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}


void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

We created a string soFar to store the binary digits we have so far

Practice: Printing binary numbers

Sample Implementation:

```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
  printAllBinaryHelper(digits – 1,soFar + “0”);
  printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}
```

```
void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

A helper function was used so that we can have a string as additional input

Practice: Printing binary numbers

Sample Implementation:

```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
    printAllBinaryHelper(digits – 1,soFar + “0”);
    printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}
```

For every recursive call, we subtract 1 from digits and we append either a 0 or 1 at the end of the soFar string.

```
void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

Practice: Printing binary numbers

Sample Implementation:

```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
  printAllBinaryHelper(digits – 1,soFar + “0”);
  printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}


void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

The base case is reached when digits is equal to 0 indicating that soFar already has n binary digits

Practice: Printing binary numbers

Sample Implementation:

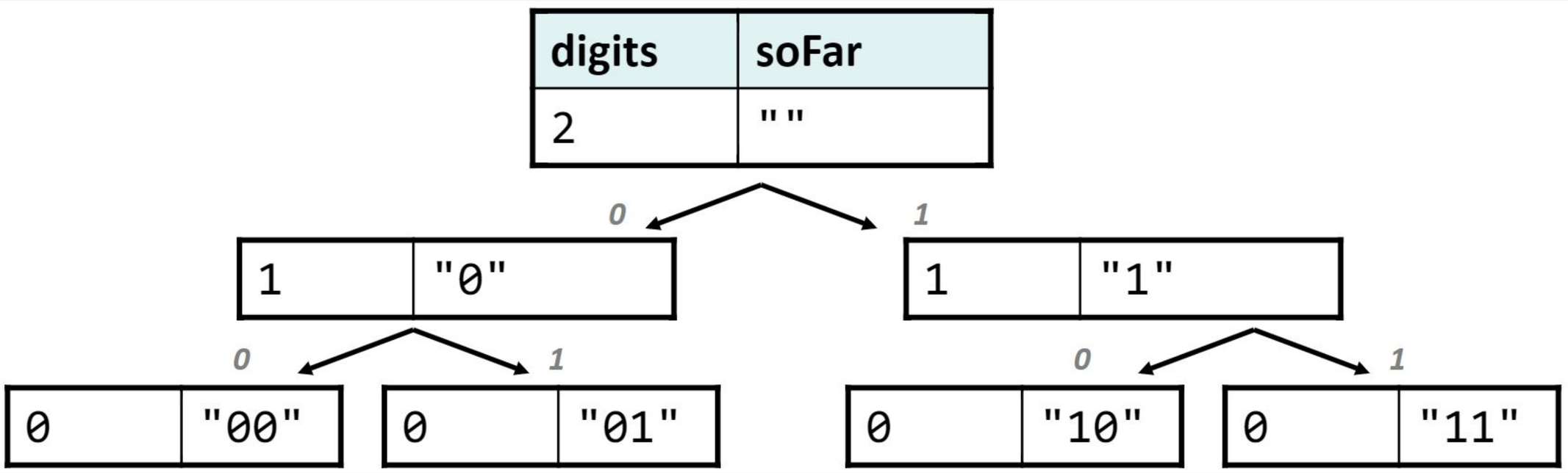
```
void printAllBinaryHelper(int digits, string soFar) {
    if(digits == 0) {
        cout << soFar << endl;
    } else {
  printAllBinaryHelper(digits – 1,soFar + “0”);
  printAllBinaryHelper(digits – 1,soFar + “1”);
    }
}
```

Question: Why do we perform the recursive call to append 0 first?

```
void printAllBinary(int numDigits) {
    printAllBinaryHelper(numDigits, “”);
}
```

Practice: Printing binary numbers

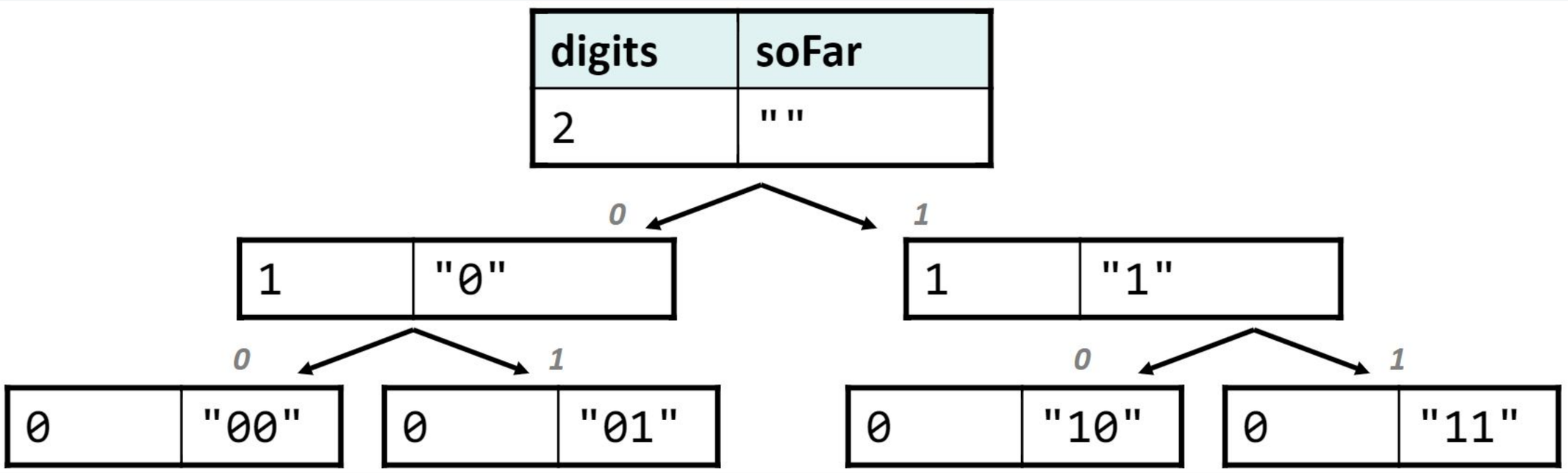
Sample Recursive Tree:



We perform the recursive call that appends 0 to the end of the string so that we can print the result in ascending order

Practice: Printing binary numbers

Sample Recursive Tree:



Can you try modifying the code such that we can print in descending order?

Practice: Dice Roll Sum

Suppose we want to create a recursive function **`diceSum()`** that accepts the number of dice rolls and desired sum as its input. The function prints only the combinations that add up exactly to that sum.

```
diceSum(2, 7);
```

```
{1, 6}
{2, 5}
{3, 4}
{4, 3}
{5, 2}
{6, 1}
```

```
diceSum(3, 7);
```

```
{1, 1, 5}
{1, 2, 4}
{1, 3, 3}
{1, 4, 2}
{1, 5, 1}
{2, 1, 4}
{2, 2, 3}
{2, 3, 2}
{2, 4, 1}
{3, 1, 3}
{3, 2, 2}
{3, 3, 1}
{4, 1, 2}
{4, 2, 1}
{5, 1, 1}
```

Practice: Dice Roll Sum

Sample Implementation:

```
void diceSumHelper(int dice, int sum, int desiredSum, vector<int> &chosen) {
     if(dice == 0) {
       if(sum == desiredSum) {
         printDice(chosen); // display content of vector
       }
     } else if(sum + 1*dice <= desiredSum && sum + 6*dice >= desiredSum) {
       for(int i = 1; i <= 6; i++) {
          chosen.push_back(i);
          diceSumHelper(dice - 1, sum + i, desiredSum, chosen);
          chosen.pop_back();
        }
     }
}

void diceSum(int dice, int desiredSum) {
     vector<int> chosen;
     diceSumHelper(dice,0,desiredSum, chosen);
}
```

Practice: Dice Roll Sum

Sample Implementation:

```
void diceSumHelper(int dice, int sum, int desiredSum, vector<int> &chosen) {
     if(dice == 0) {
       if(sum == desiredSum) {
         printDice(chosen); // display content of vector
       }
     } else if(sum + 1*dice <= desiredSum && sum + 6*dice >= desiredSum) {
       for(int i = 1; i <= 6; i++) {
          chosen.push_back(i);
          diceSumHelper(dice - 1, sum + i, desiredSum, chosen);
          chosen.pop_back();
       }
     }
}

void diceSum(int dice, int desiredSum) {
     vector<int> chosen;
     diceSumHelper(dice,0,desiredSum, chosen);
}
```

For each possible value of a die (1-6), we perform a recursive call.

Practice: Dice Roll Sum

Sample Implementation:

```
void diceSumHelper(int dice, int sum, int desiredSum, vector<int> &chosen) {
     if(dice == 0) {
       if(sum == desiredSum) {
         printDice(chosen); // display content of vector
       }
     } else if(sum + 1*dice <= desiredSum && sum + 6*dice >= desiredSum) {
       for(int i = 1; i <= 6; i++) {
          chosen.push_back(i);
          diceSumHelper(dice - 1, sum + i, desiredSum, chosen);
          chosen.pop_back();
       }
     }
}

void diceSum(int dice, int desiredSum) {
     vector<int> chosen;
     diceSumHelper(dice,0,desiredSum, chosen);
}
```

If the current sum is equal to the desired sum, the vector is printed,

The vector is used to store the combinations of the die rolls.

Practice: Dice Roll Sum

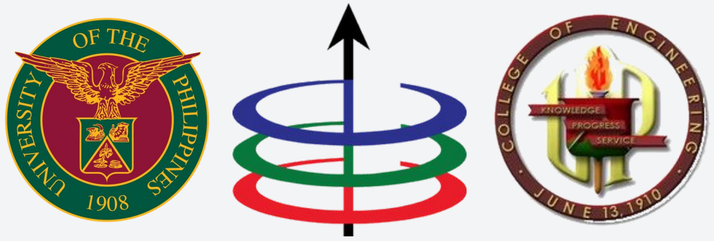
Sample Implementation:

```
void diceSumHelper(int dice, int sum, int desiredSum, vector<int> &chosen) {
     if(dice == 0) {
       if(sum == desiredSum) {
         printDice(chosen); // display content of vector
       }
     } else if(sum + 1*dice <= desiredSum && sum + 6*dice >= desiredSum) {
       for(int i = 1; i <= 6; i++) {
          chosen.push_back(i);
          diceSumHelper(dice - 1, sum + i, desiredSum, chosen);
          chosen.pop_back();
       }
     }
}

void diceSum(int dice, int desiredSum) {
     vector<int> chosen;
     diceSumHelper(dice,0,desiredSum, chosen);
}
```

This part of the code prunes the possible cases to reduce recursive calls.

Basically, if the current sum is outside the range of the remaining die rolls, we don't expand the tree anymore.

Any questions?



Lecture 3
Analyzing Recursive Functions

EEE 121 2s2526

Steven S. Sison